StreetCare AI

An AI-assisted review platform for pothole and street defect reports. It cleans historical report data, predicts how likely a defect is to come back and how long its repair will take, screens every open report against the council's defect standards, and uses AI agents to explain each case so that a Highways Inspector can take the repair decision.

TEAM MEMBERS

Mansavi Aggarwal

Puttamaneni Srinivasa Praneeth

Riya Rajeev Shinde

Bhavesh Durai S

Paravthy Vinod K V

Document information

Item	Detail
Project	StreetCare AI — pothole and street defect report review platform
Client (scenario)	Wrenford Borough Council, Street Care Office
Document	Project report: requirements, design, data, models, agents, service, testing, delivery and operations
Repository state	branch `main`, commit `1df091a`; all modules implemented, no empty placeholders
Audience	project reviewers and assessors, the delivery team, engineers who will maintain the system
Source of figures	every number is measured from the project's own outputs: `output/data_quality_report.json`, `output/model_metrics.json`, `output/model_benchmark_comparison.json`, the cleaned CSVs, a fresh end-to-end build and a captured service run

Abstract

Residents report potholes, broken paving, blocked drains, overgrown vegetation and failed street lights through a website, an app and a contact centre. Each report has to be checked against the council's defect standards, weighed against street works on the same street and turned into one of three decisions: **order the repair**, **inspect on site** or **refer for coordination**. StreetCare AI supports that work end to end.

A data pipeline cleans five historical extracts (110,172 rows → 105,808) whose answers arrive in mixed spellings and units. Three scikit-learn models are trained on them: a repeat-likelihood classifier (held-out ROC AUC **0.8351**), an expected-crew-hours regressor (R^2 **0.9025**) and a sentence-type text classifier (macro F1 **0.9775**). Sixteen PDFs are split into 205 passages and indexed in ChromaDB, and the requirement thresholds are read from those PDFs at run time rather than written in code.

At run time every open report is screened deterministically. A LangGraph workflow then runs five Gemini agents — a planner, a requirements analyst, an evidence analyst, a risk analyst and a writer — which explain the findings and recommend a decision. The run pauses, durably, until a Highways Inspector appointed for the street's district (and who did not open the review) decides; up to two further agents then write the works order or inspection request and the resident update. A retrieval-grounded chat answers staff questions from the documents, while questions about a single report or street are answered by code under role- and privacy-based rules. A FastAPI service with session tokens, role permissions and district checks serves a Streamlit dashboard. Fifty automated tests check the result; all fifty pass with the language model available.

Contents

1 Introduction and problem statement

2 Requirements and design principles

3 System architecture

4 Technology stack and code structure

5 The data

6 Exploratory data analysis

7 Data cleaning pipeline

8 Feature engineering

9 Machine learning models

10 Model benchmarking

11 Knowledge base and retrieval

12 Screening and business rules

13 The agentic review workflow

14 The street care chat

15 Persistence and data model

16 REST API and security

17 The dashboard

18 Testing and quality assurance

19 Delivery plan and team organisation

20 Deployment and operations

21 Limitations, risks and future work

22 Conclusion

A Configuration reference

B API reference with examples

C File inventory

D Glossary

CHAPTER 1

Introduction and problem statement

Why the Street Care Office needs help, and what the system must and must not do

1.1 The business problem

Wrenford Borough Council receives defect reports about its streets from three channels: the council website, the StreetCare mobile app and the contact centre. A report names the street and a location on it, the defect type (carriageway, footway, drainage, vegetation or street lighting), sometimes a second, linked defect type, and answers to the questions on the reporting form: depths and widths, standing water, clear heights under vegetation, lamp column details, whether a photograph is attached, and how often the location has been reported before. Documents may be attached: a resident's description, a call note, a photo note or an inspection note.

The Street Care Office must decide, for every report, between three outcomes:

Decision	Meaning	What is written
ORDER REPAIR	the repair goes ahead from the report as it stands	a works order to the council's repair contractor, then an update for the resident
INSPECT ON SITE	an inspector visits and measures before anything is repaired	an inspection request listing what must be settled
REFER FOR COORDINATION	the network coordination team fits the repair around street works, traffic or the platform it needs	a coordination brief built from the review

Doing this by hand is slow and inconsistent. Each defect type has its own standard with its own thresholds; forms are filled in with inconsistent spellings and units; street works permits on the same street change the right answer; and the attached documents sometimes disagree with the form. Two questions matter for planning but cannot be read off a form at all: *will this defect come back after it is repaired?* and *how many crew hours will it really take?*

1.2 Objectives

1. Clean and profile the historical data so that it can be trusted, with every change explained.
2. Predict the likelihood that a repaired defect is reported again, and the crew hours a repair will take.
3. Classify the sentences of attached documents so that evidence can be set against the report.
4. Screen each report against its own defect standard, read from the council's documents, not from code.
5. Use language-model agents to **explain** the screening, the evidence and the predictions and to **recommend** — never to decide.
6. Make sure a Highways Inspector appointed for the street's district, other than the person who opened the review, decides.
7. Write the documents each decision calls for, and keep a complete audit trail.
8. Give staff a chat assistant that answers from the documents and respects what each role may be told.

1.3 Scope

In scope	Out of scope

Data cleaning and quality profiling · three ML models · PDF knowledge base and retrieval · screening rules · LangGraph agent workflow · chat assistant · SQLite persistence · FastAPI service with authentication and authorisation · Streamlit dashboard · automated tests	Collecting reports from the public · dispatching crews · contractor systems · payment · live integration with a street works register API (a CSV extract is used) · mobile apps

1.4 Stakeholders and roles

Role	Accounts	What they do in the system
Highways Inspector	8 (two per district)	work the queue, open reviews, **decide** reviews on their own district's streets, read the audit trail, use the chat
Street Care Coordinator	4	work the queue, open reviews anywhere, read the audit trail, use the chat — but never decide
Customer Services Adviser	4	chat only: tell a caller a report's status and, once a repair is ordered, what the resident was told — never a prediction

1.5 Structure of this report

Chapters 2–4 set out the requirements, architecture and technology. Chapters 5–10 cover the data, its analysis, cleaning, features, models and benchmarking. Chapters 11–14 describe the knowledge base, the screening rules, the agent workflow and the chat. Chapters 15–17 cover persistence, the API and the dashboard. Chapters 18–22 report testing, delivery, operations, limitations and conclusions. The appendices give a configuration reference, API examples, a file inventory and a glossary.

CHAPTER 2

Requirements and design principles

The rules the specification sets, and the principles the design follows

2.1 Functional requirements

ID	Requirement	Where it is met
F1	Clean five historical extracts and write a data-quality report	`data_pipeline/*`, `run_data_pipeline.py`
F2	Train and save a repeat-likelihood classifier, an expected-crew-hours regressor and a sentence-type classifier, each above its minimum	`modeling/*`, `run_model_training.py`
F3	Read every requirement of each defect standard from its PDF at run time	`knowledge/standard_reading.py`
F4	Index the sixteen PDFs for section reads and search	`knowledge/*`, `run_knowledge_indexing.py`
F5	Rank the open queue (footway first) and screen each report: requirements, permits, triggers, assessment, decisions open	`tools/report_data_tools.py`, `tools/screening_tools.py`
F6	Classify document sentences, link them to requirements and flag figures that disagree with the form	`tools/evidence_tools.py`
F7	Run five reviewing agents, store the review, pause for an inspector and resume with the decision	`agentic/*`
F8	Write a works order + resident update, an inspection request, or a coordination brief	`agentic/agents.py`, `tools/review_document_tools.py`
F9	Answer chat questions from documents, records or refuse; record every turn	`chat/*`
F10	Sign in / out, role permissions, district rules, audit of every model call	`access/*`, `api/routes.py`, `persistence/*`
F11	A dashboard with pages per role	`dashboard/*`, `run_dashboard.py`

2.2 Non-functional requirements

Area	Requirement
No hard-coding	every path, threshold, mapping, message and name lives in `config/config.py`; requirement figures are read from the PDFs
Exactness	readings converted with Python `Decimal` and rounded half-up; thresholds compared exactly
Refuse, never guess	missing identifiers, repeated keys, unreadable values, a screening of the wrong report, a malformed agent reply → `ValueError`
Determinism	every figure shown to a person (screening, permits, likelihood, crew hours) comes from code and saved models; agents only explain
Durability	a paused review survives a restart (SQLite checkpoint); decisions are written in one transaction
Cost control	exactly 5 model calls before the gate and 7 / 6 / 5 in all; every refusal happens before a call is spent

Security	salted PBKDF2 password hashes; one session per account; role and district checks in the service, not the dashboard
Privacy	advisers are never told predictions; street figures below five reports are withheld
Testability	50 automated tests run against the real service, models and data

2.3 Design principles

Code decides, AI explains

All rules — screening, assessment, which decisions are open, who may decide — are deterministic code. The language model is used where language is the job: explaining, recommending and drafting documents.

Documents are the source of truth

Thresholds such as “a depth of at least N millimetres” are parsed from section 4 of each standard. Changing the PDF changes the rule without a code change.

One source of configuration

`config.py` (1,393 lines) holds every constant, so a reviewer can audit the system's behaviour in one place and no module hides a magic number.

Fail loudly and early

Bad data is refused at the boundary with a message naming the row and column. A request the rules would refuse is refused before a model call or a database write.

Human in the loop, by construction

The workflow physically cannot record a decision without an inspector's input passing every check in `find_decision_refusal`.

Everything audited

Every run, review, decision, model call (tokens and latency) and chat turn is stored and visible in the audit trail.

CHAPTER 3

System architecture

Six layers from supplied files to the person who decides

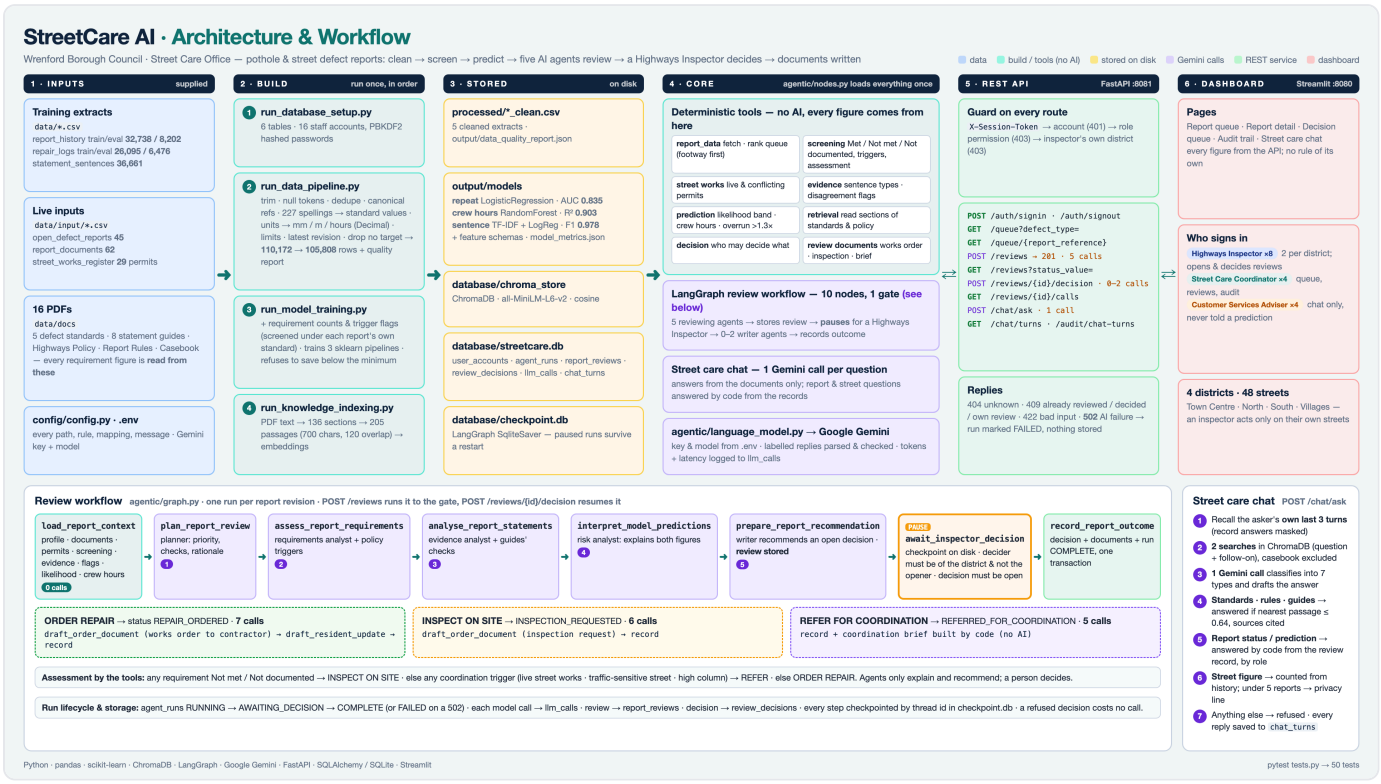


Figure 3.1 · Layered architecture: inputs → build → stored artefacts → core → REST API → dashboard, with the review workflow and chat underneath

3.1 Layers

1. **Inputs** — five training CSVs, three live CSVs (open reports, their documents, the street works register), sixteen PDFs and `config.py` with `.env`.
2. **Build** — four scripts run once, in order: database setup, data pipeline, model training, knowledge indexing.
3. **Stored artefacts** — cleaned CSVs, three joblib models with JSON input schemas and metrics, a ChromaDB store, the application database and the LangGraph checkpoint database.
4. **Core** — `agentic/nodes.py` loads the live files, models, documents and index once. Deterministic tools compute every figure; the LangGraph workflow and the chat call Gemini through one client.
5. **REST API** — FastAPI on port 8081. One guard on every route; the queue and every report detail are computed once at start-up.
6. **Dashboard** — Streamlit on port 8080; holds no business rule and reads no file of its own.

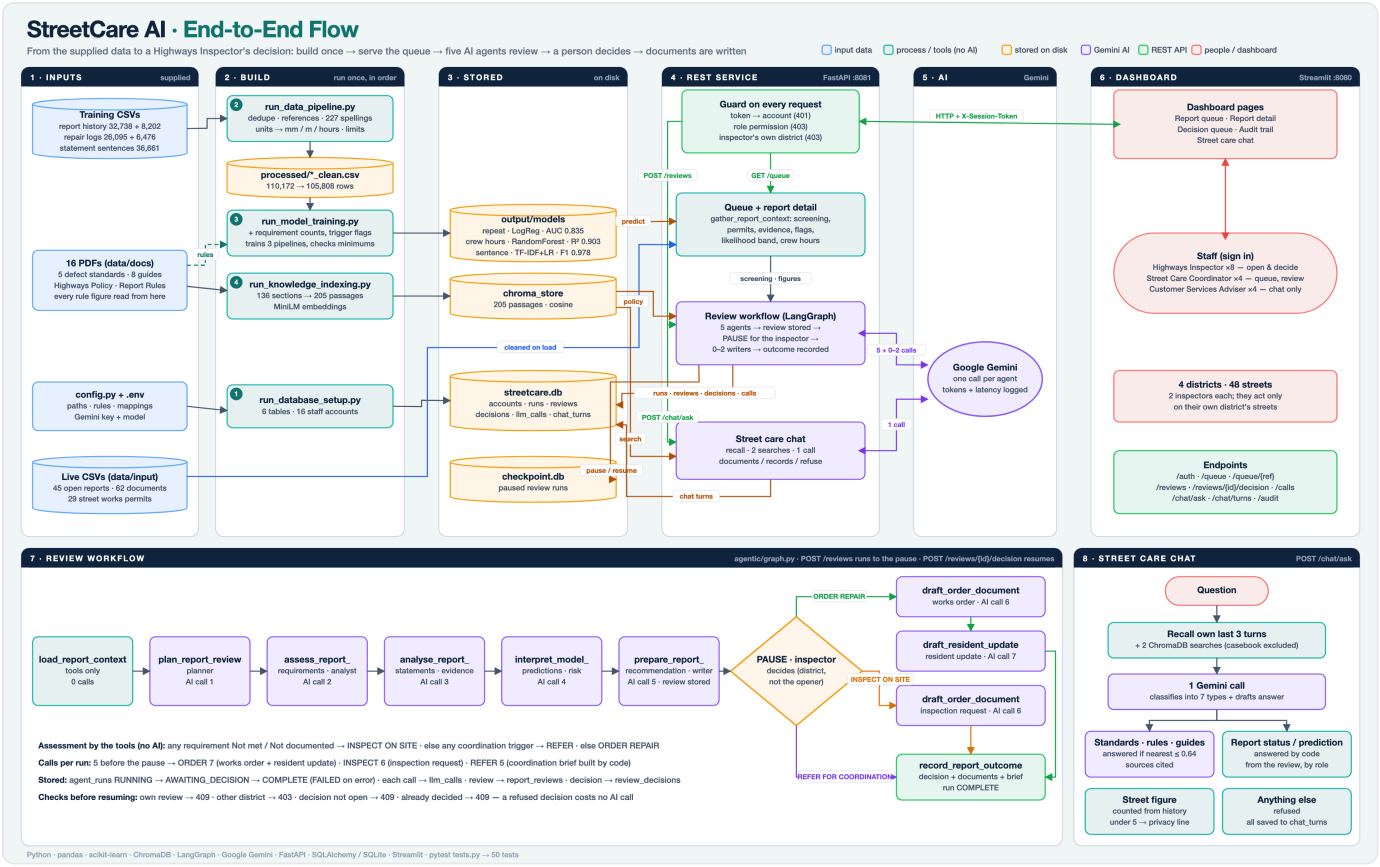


Figure 3.2 · End-to-end flow: data and build (top left), the service and its stores (top right), the review workflow and the chat (bottom)

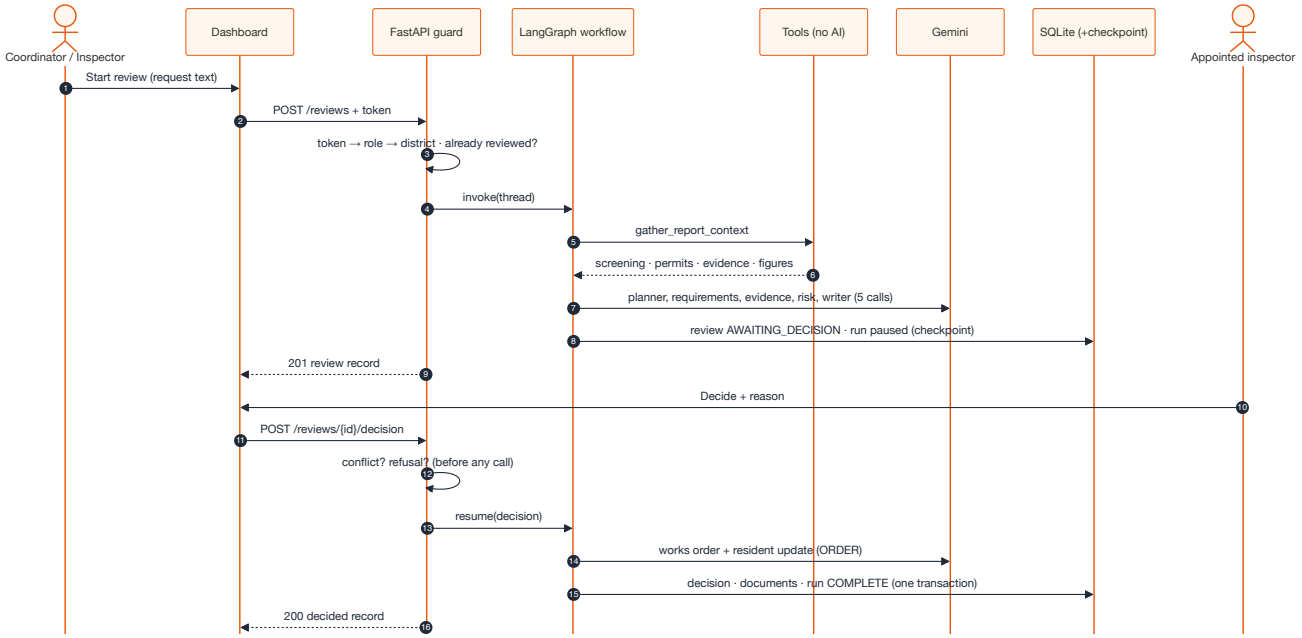


Figure 3.3 · A review from start to decision, as a sequence

3.2 Why this shape

The split between a deterministic core and an explaining AI layer is the main architectural decision. It means the numbers a person relies on are reproducible and testable, the AI cannot invent a figure, and the system degrades safely: if Gemini is unavailable the queue, the detail, the screening and the predictions still work, and only the steps that need language fail with a clear 502. Computing the queue once at start-up keeps every page fast and guarantees the queue, the detail and the stored review agree, because they all come from the same call to `gather_report_context`.

CHAPTER 4

Technology stack and code structure

What the system is built with, and where each piece lives

4.1 Technology stack

Layer	Technology	Used for
Language	Python 3.11–3.14	everything
Data	pandas 3, NumPy, Decimal	cleaning, profiling, exact unit conversion
Machine learning	scikit-learn 1.9 (LogisticRegression, RandomForestRegressor, TfidfVectorizer, ColumnTransformer), joblib	three model pipelines, persistence
Documents	pypdf	text extraction from the 16 PDFs
Retrieval	ChromaDB, sentence-transformers all-MiniLM-L6-v2, cosine distance	passage index, section reads, chat search
Agents	LangGraph (StateGraph, interrupt, SqliteSaver), google-genai	the review workflow and its durable pause; Gemini calls
Persistence	SQLAlchemy 2 ORM on SQLite	accounts, runs, reviews, decisions, model calls, chat turns
Service	FastAPI, Pydantic 2, Uvicorn	REST API, validation, error handling
Dashboard	Streamlit, httpx	five pages and sign-in
Testing	pytest, FastAPI TestClient	50 marking tests

4.2 Folder layout

```
Project/
├── config/config.py            every setting
├── data/                       5 training CSVs · input/ (3 live CSVs) · docs/ (16 PDFs)
├── data_pipeline/              loader · common cleaning · 6 cleaners · quality profiler · feature
builder
├── modeling/                   repeat · crew hours · sentence · evaluation · registry
├── knowledge/                  pdf_extraction · chunk_splitting · vector_index · standard_reading
├── tools/                      report data · screening · evidence · prediction · retrieval · decision
· documents
├── persistence/                database · models (6 tables) · repository
├── access/                     password_security · permissions
├── agentic/                    state · language_model · agents · nodes · graph
├── chat/                       conversation_memory · street_care_assistant · record_answer
├── api/                        schemas · routes
├── dashboard/                  api_client · sign_in · 5 pages
├── processed/ · output/ · database/     generated by the build steps
├── run_database_setup.py · run_data_pipeline.py · run_model_training.py · run_knowledge_indexing.py
├── run_api_service.py · run_dashboard.py · experiment_models.py
└── tests.py                    50 tests
```

CHAPTER 5

The data

Thirteen CSV files, sixteen PDFs, and how they relate

5.1 Data sources

File	Rows raw	Rows clean	Cols	Target
`data/report_history_train.csv`	32,738	30,569	31	reported_again
`data/report_history_eval.csv`	8,202	7,648	31	reported_again
`data/repair_logs_train.csv`	26,095	25,537	27	actual_crew_hours
`data/repair_logs_eval.csv`	6,476	6,337	27	actual_crew_hours
`data/statement_sentences.csv`	36,661	35,717	8	sentence_type
`data/input/open_defect_reports.csv`	45	40	27	—
`data/input/report_documents.csv`	62	60	6	—
`data/input/street_works_register.csv`	29	28	6	—

The **report history** holds decided reports, one row per revision, with what happened next — including whether the defect was **reported again** (the repeat model's target). The **repair logs** hold repaired reports with the **actual crew hours** (the crew hours model's target). The **statement sentences** are labelled sentences from documents attached to older reports, with nine **sentence types**. The three **live** files are the open queue, its documents and the street works register; they are cleaned in memory each time the service starts.

5.2 Report schema (shared by history and the live queue)

Group	Columns
Identity	`report_reference`, `revision_number`, `report_channel`, `reporter_type`
Place	`street_name` (48 streets), `district` (4), `street_category`, `location_description`, `location_fixed`
Defect	`defect_type`, `linked_defect_type`, `photograph_attached`
Readings	`defect_depth_mm`, `defect_width_mm`, `water_depth_mm`, `water_location`, `water_reach_m`, `overhang_location`, `clear_height_m`, `on_highway_land`, `lighting_fault`, `column_number_given`, `column_lean_degrees`, `column_height_m`
Context	`previous_reports_at_location`, `planned_crew_hours`, `live_street_works_on_street`, `conflicting_street_works`
History only (outcomes)	`review_outcome`, `reported_again` (target), `repeat_cause`
Live only	`supporting_document_count`; the two street works counts are computed from the register at load

Repair logs add `repair_method`, `traffic_management`, `crew_size`, `completion_code` and the target `actual_crew_hours`. Statement sentences hold `sentence_id`, `report_reference`, `report_channel`, `document_type`, `sentence_number`, `sentence_text`, `sentence_length_characters` and the target `sentence_type`. Full per-column schemas, with profiles, are in the companion *CSV Schemas* PDF and the *Data Knowledge Workbook*.

5.3 How the files relate

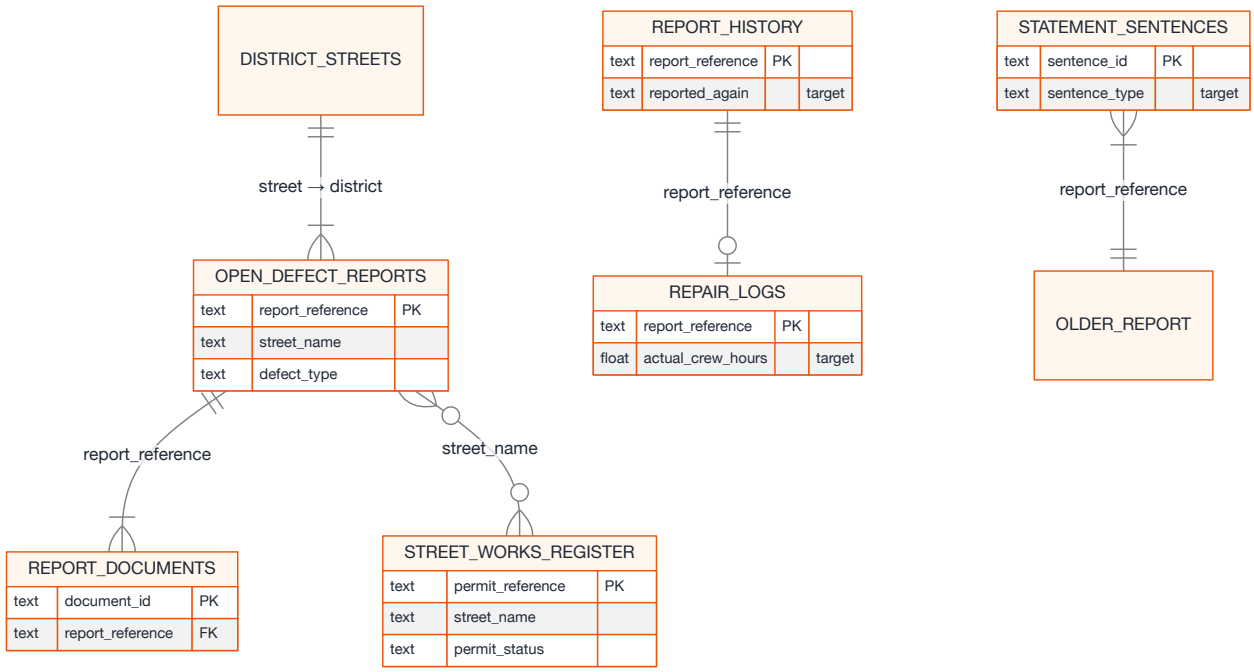


Figure 5.1 · Logical relationships between the data files

Link	Measured on the real data
report_history_train → report_history_eval	0 shared references
repair_logs_train → repair_logs_eval	0 shared references
report_history_* → repair_logs_*	31644 of 31874 log references are in the 38217 history references; by split: train history × train logs 25352, train history × eval logs 0, eval history × train logs 0, eval history × eval logs 6292
statement_sentences → report_history_*	5902 reports, median 6 sentences per report; 0 of them in the history extracts
report_history_* (raw) → report_history_* (clean)	duplicates and older revisions removed in cleaning
open_defect_reports → report_documents	60 documents for 40 of 40 reports; 0 reports have none; 0 documents point at no open report; supporting_document_count matches in 40 of 40
open_defect_reports → street_works_register	28 permits (23 Live) on 26 streets; 19 of 40 reports are on a street with a live permit
open_defect_reports → report_history_*	0 shared references
report_history_train → config.INSPECTION_DISTRICT_STREETS	0 rows whose district disagrees with the street's
repair_logs_train → config.INSPECTION_DISTRICT_STREETS	0 rows whose district disagrees with the street's

open_defect_reports → user_accounts (Highways Inspector)	0 live reports whose district disagrees with the street's

5.4 Data quality defects

Every defect below was counted by the quality profiler on the raw extracts before cleaning (`output/data_quality_report.json`).

Defect found in the raw data	History train	History eval	Logs train	Logs eval	Sentences
Rows received	32,738	8,202	26,095	6,476	36,661
Exact duplicate rows	369	93	303	74	445
References not written canonically	18,705	4,727	14,893	3,741	20,081
Category values not written canonically	150,500	38,140	115,903	29,101	4,300
Readings not in the standard unit	38,069	9,544	40,306	10,012	—
Blank cells as received	341,357	85,523	198,222	49,283	283
Values below the minimum	205	49	146	39	—
Values above the maximum	3	0	0	0	—
Earlier revisions of a report	1,565	408	—	—	—
Rows with no target answer	235	53	209	49	283
Rows with an impossible target	—	—	46	16	—
Sentences with HTML markup / in capitals	—	—	—	—	3,560 / 501
Rows removed in total	2,169	554	558	139	944
Rows kept	30,569	7,648	25,537	6,337	35,717

5.5 Results

The checks show the data can be trusted once cleaned. The train and evaluation files share no report, so the held-out scores are honest, every open report has its documents, and every street sits in the right district. The quality report shows that most defects are presentation problems rather than missing information: references written three ways, categories spelt differently and readings in mixed units, all resolved by cleaning without losing a report. The 4,364 rows removed are exact duplicates, superseded revisions or rows with no usable target, and each is counted with its reason.

CHAPTER 6

Exploratory data analysis

What the cleaned data says about repeats, crew hours and documents

The analysis below uses the cleaned training and evaluation extracts together (38,217 decided reports, 31,874 repair logs, 35,717 sentences). It explains why the models use the features they do and what a reviewer should expect.

6.1 Which defects come back

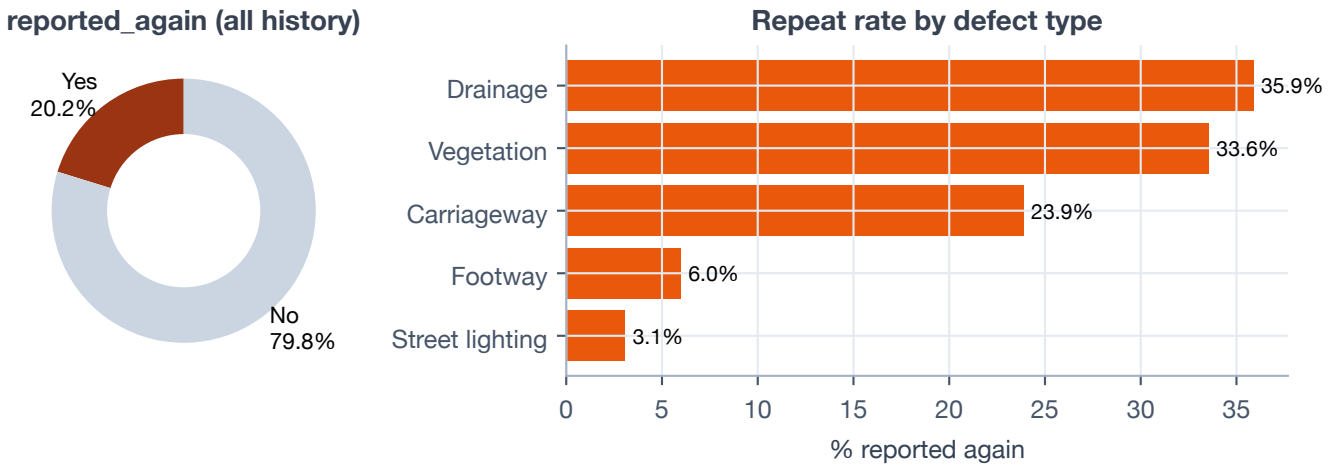


Figure 6.1 · 20.23% of repaired defects were reported again; the rate varies tenfold by defect type

Drainage (35.9%) and vegetation (33.6%) defects recur most: gullies block again and vegetation regrows. Footway (6.0%) and street lighting (3.1%) repairs tend to hold. The class balance (about one in five) is why the repeat model is judged by ROC AUC and Brier score rather than accuracy: a model that always said “no” would be 80% accurate and useless.

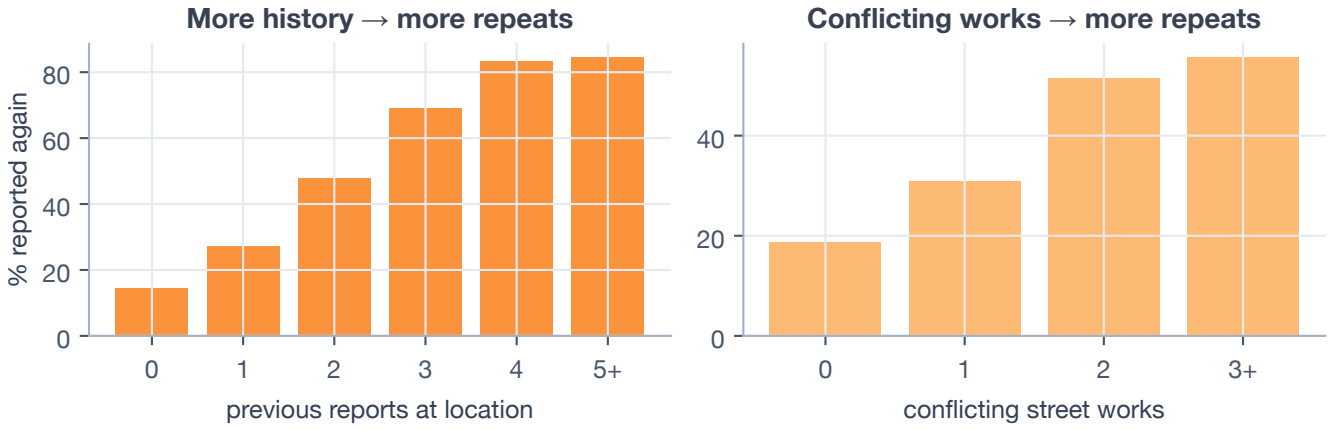


Figure 6.2 · Repeat rate by previous reports at the location and by conflicting street works

Two context fields are the strongest signals. A location with three earlier reports was reported again 69.0% of the time against 14.3% for a first report; conflicting street works on the same highway part roughly triple the rate. Both are used as features, and conflicting works also raise the *live street works* coordination trigger.

6.2 How long repairs take

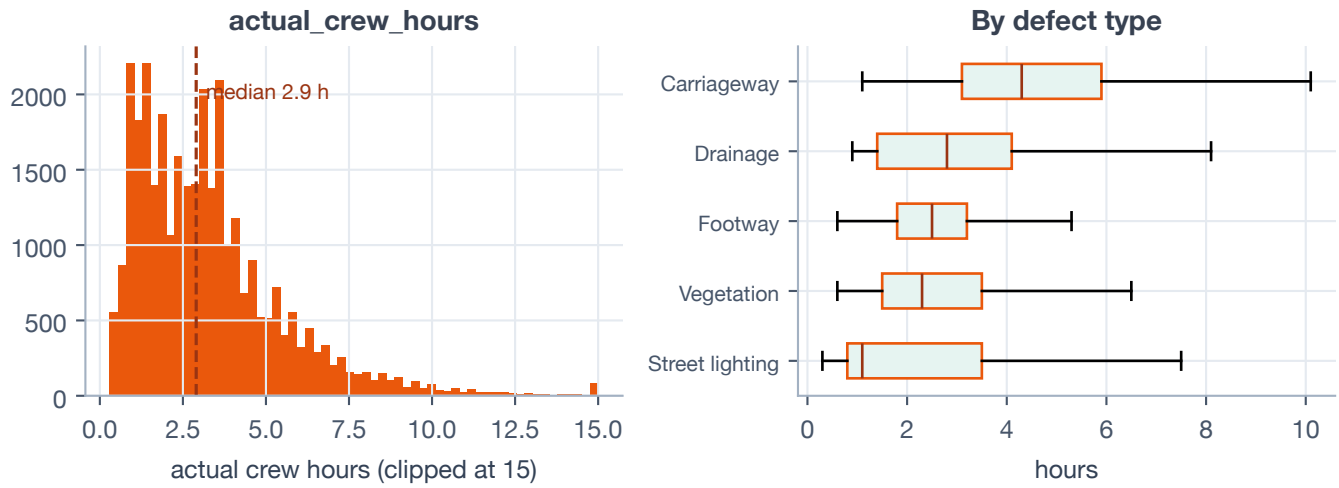


Figure 6.3 · actual_crew_hours is right-skewed; carriageway repairs take longest

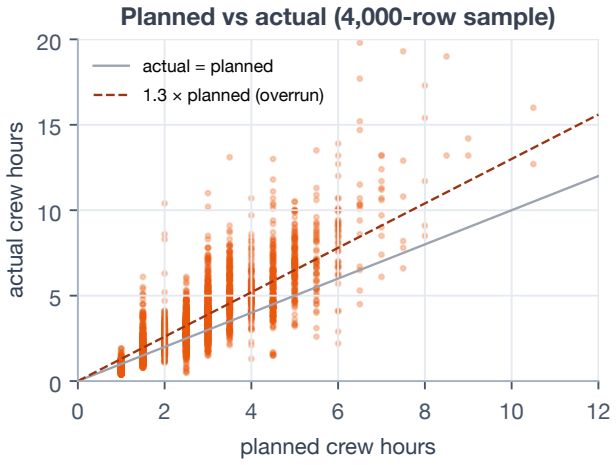


Figure 6.4 · Planned vs actual: 36.5% exceed 1.3 × planned

Actual hours track planned hours but with wide spread and a long tail (maximum 43.9 h). A random forest captures the non-linear interactions between defect type, street category, triggers and planned hours far better than a linear model (Chapter 10). An *expected overrun* is flagged when the predicted hours exceed 1.3 × planned — a threshold the council sets in configuration — so a coordinator can plan traffic management and crew size before the job starts.

6.3 What the documents say

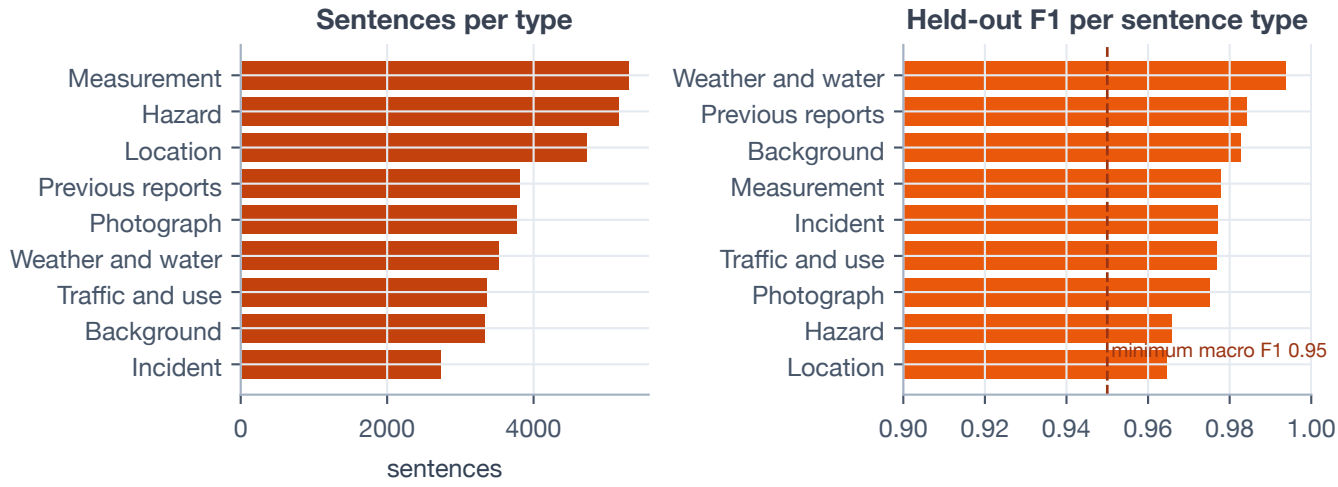


Figure 6.5 · Nine sentence types are reasonably balanced (7.6%–14.9%), and each is classified above 0.96 F1

Because no class dominates, macro F1 (the unweighted mean across classes) is a fair summary. Measurement, Location and Photograph sentences matter most: they bear on specific requirements, and Measurement sentences are compared figure-by-figure with the form to flag disagreements.

6.4 Data quality highlights

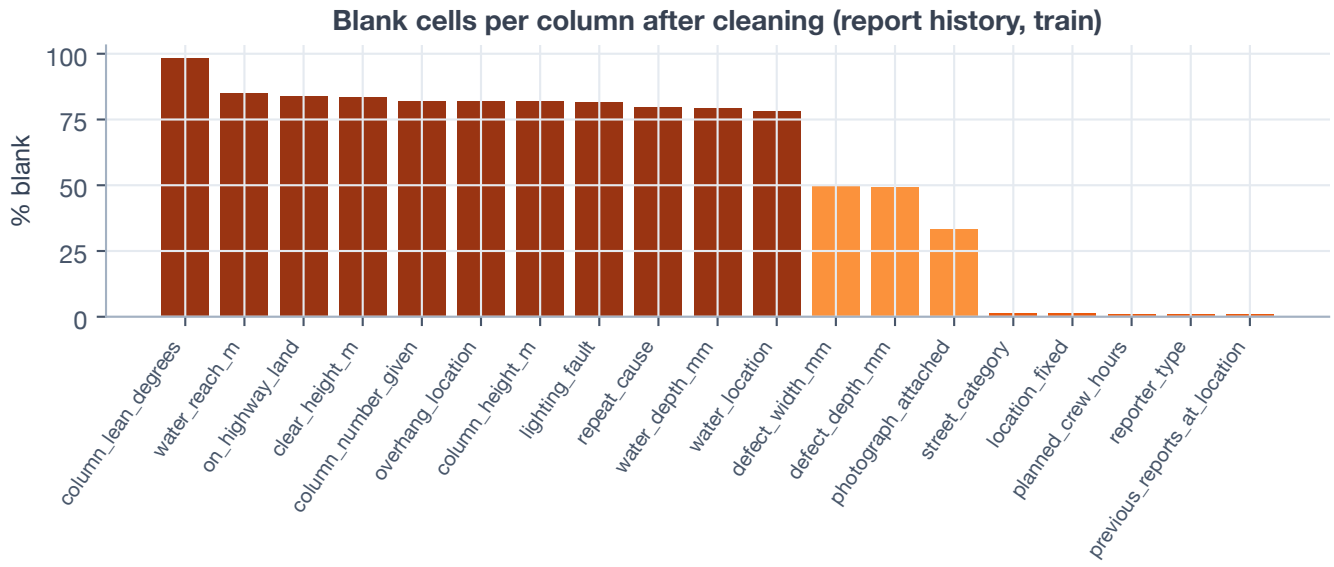


Figure 6.6 · Blank cells after cleaning: high for readings that only apply to one defect type

High blank rates are expected, not a fault: `column_lean_degrees` applies only to leaning lamp columns and `clear_height_m` only to vegetation. Blank stays blank in the cleaned files; the model pipelines fill numeric gaps with the training median and categorical gaps with the most frequent value, inside the pipeline so the same rule applies at prediction time.

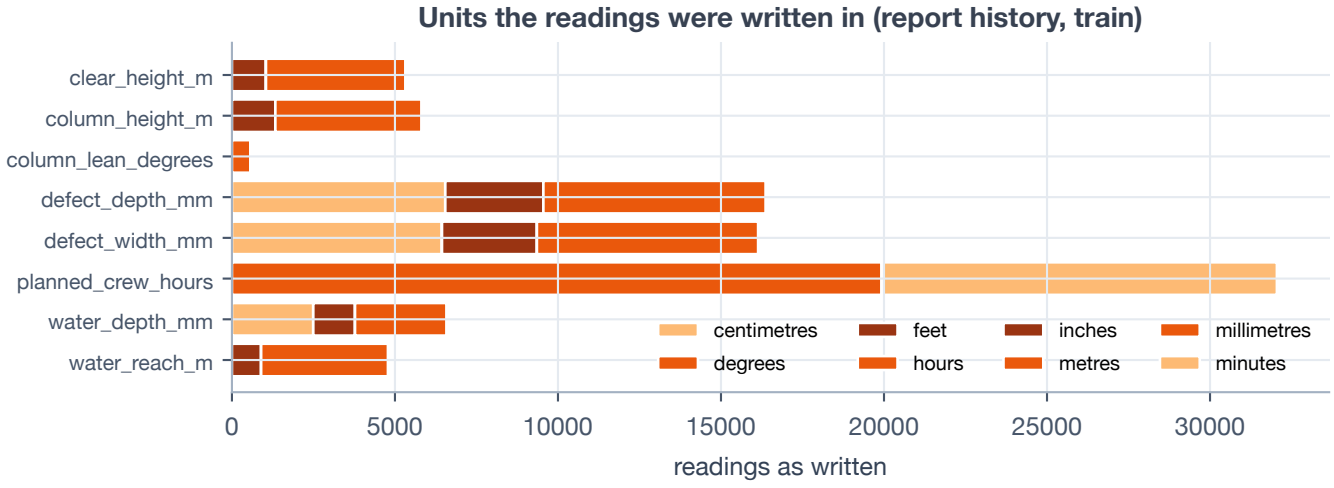


Figure 6.7 · The same column arrives in different units; all are converted to one unit with exact Decimal arithmetic

CHAPTER 7

Data cleaning pipeline

From messy forms to trusted tables, with every change counted

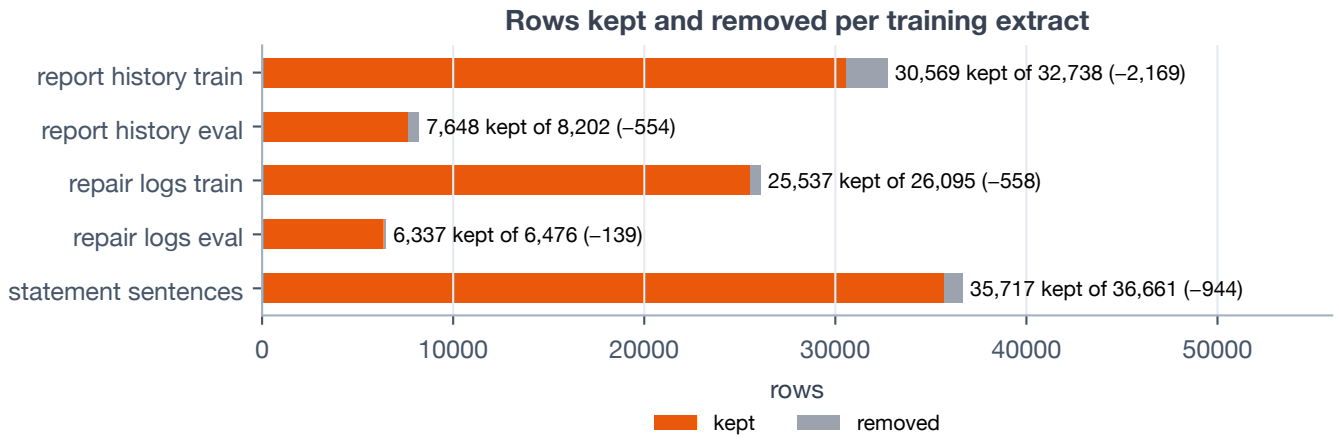


Figure 7.1 · Rows received and kept per extract (4,364 rows removed in total)

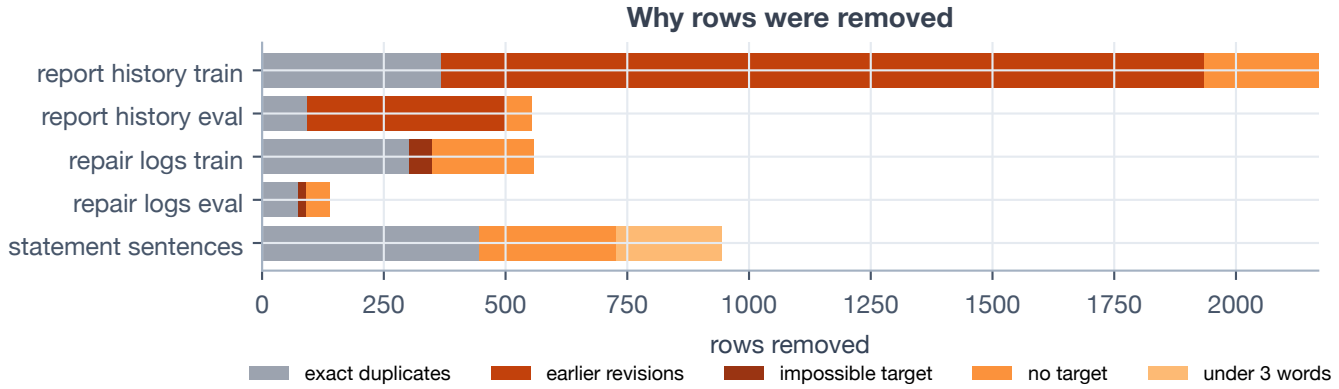


Figure 7.2 · Every removed row has a reason

7.1 Steps, in order

#	Step	Applies to	Effect on report history (train)
1	Read every column as text, trim, turn null tokens (`n/a, na, –, null, not known, caller unsure`) into blanks	all	345,245 blank cells as read
2	Remove exact duplicate rows	all	369 rows
3	Canonical report reference `SC` + 7 digits (`SC–52125` , `52125` → `SC0052125`)	all	18,705 references rewritten
4	Fold spelling variants onto canonical categories (227 variants)	categoricals	150,500 values rewritten
5	Whole numbers by pattern; refuse unreadable	counts, revisions	—
6	Readings with units → one unit, rounded half-up (mm whole; m, degrees, hours to 0.1)	8–9 reading columns	87,665 readings converted

7	Below minimum / above maximum → blank	readings, counts	205 below, 3 above
8	Keep only the latest revision of each report (history and live)	history, live	1,565 earlier revisions
9	Remove rows with no target (history, logs) or an impossible one	training extracts	235 rows
10	Sentences: strip markup, decode entities, lower ALL-CAPS, drop under 3 words, recompute length	sentences	3,560 with markup, 501 in capitals, 216 dropped

7.2 Refuse, never guess

Some problems cannot be fixed safely and stop the pipeline with a message naming the column and the values: a missing report or sentence identifier, an identifier that does not match its pattern, the same repair log twice with different content, two rows for the same report revision, or a blank revision. Silently dropping or merging such rows would hide a problem in the source system, so the pipeline makes it visible.

7.3 Exact unit conversion

Reading	Written as	Converted to	Example
depth, width, water depth	45 · 4.5 cm · 1.75 in	whole millimetres (×10 cm, ×25.4 in)	1.75 in → 44 mm; 0.8 in → 20 mm
reach, clear height, column height	1.2 · 1.2 m · 4 ft	metres to 0.1 (×0.3048 ft)	4 ft → 1.2; 26 ft → 7.9
column lean	6.5 · 6.5 deg	degrees to 0.1, 0–90	4.95 → 5.0
planned / actual crew hours	2.5 · 2.5 hrs · 150 min	hours to 0.1 (÷60 min)	150 min → 2.5; 45 min → 0.8

Conversions use `Decimal` with `ROUND_HALF_UP`, so 3.95 cm becomes 40 mm and 3.94 cm becomes 39 mm, exactly as the tests require; binary floating point would round some of these the wrong way.

7.4 The quality report

`output/data_quality_report.json` records, per extract: rows received, removed and kept; duplicates; references and category values not written canonically; readings by unit; blank cells before and after; values outside limits; removals by reason; and the target's shares. It is the evidence behind every number in this chapter and the *4_Data_Quality* sheet of the workbook.

CHAPTER 8

Feature engineering

Turning a cleaned report into the exact inputs each model expects

8.1 Engineered features

Feature	How it is made	Why
`requirements_met_count`, `requirements_not_met_count`, `requirements_not_documented_count`	screen each history row under the standards of its own defect type and linked defect type, read from the PDFs	how complete and serious the report is, in the council's own terms
`live_street_works_trigger`, `traffic_sensitive_trigger`, `high_column_trigger`	0/1 flags from the same trigger rules the screening uses	coordination complexity drives both repeats and hours

Building the same features at training and prediction time from the same functions (`add_requirement_counts`, `add_coordination_trigger_flags`) removes the most common source of training/serving skew.

8.2 Model inputs

Model	Numeric inputs	Categorical inputs
Repeat likelihood (27)	depth, width, water depth, water reach, clear height, column lean, column height, previous reports, planned crew hours, live and conflicting works, 3 requirement counts, 3 trigger flags	defect type, linked defect type, street category, location fixed, photograph attached, water location, overhang location, on highway land, lighting fault, column number given
Expected crew hours (20)	the same readings, previous reports, live and conflicting works, planned crew hours, 3 trigger flags	defect type, linked defect type, street category, water location, overhang location, lighting fault
Sentence type	—	the cleaned sentence text (TF-IDF)

8.3 Preprocessing inside the pipeline

```
ColumnTransformer
├── numeric:     SimpleImputer(strategy="median")        → StandardScaler()
└── categorical: SimpleImputer(strategy="most_frequent") → OneHotEncoder(handle_unknown="ignore")
```

The input column order is saved as JSON (`repeat_features.json`, `crew_hours_features.json`). At prediction time the live report is built into exactly that frame; any mismatch raises an error rather than silently mis-aligning columns.

Columns known only after the decision or the repair — `review_outcome`, `repeat_cause`, `completion_code`, `repair_method`, `traffic_management`, `crew_size` — are never model inputs. Identifiers and free text (`report_reference`, `location_description`, `reporter_type`) are excluded too. A test checks the saved schemas leave them out.

CHAPTER 9

Machine learning models

Three models, each scored on data it never saw, each above its minimum



Figure 9.1 · Train and held-out scores against each model's minimum (dashed)

9.1 Repeat likelihood — logistic regression

A regularised logistic regression (`max_iter=3000` , seed 1313) predicts the probability that a repaired defect is reported again. It was chosen for calibrated probabilities and explainability: the risk analyst agent can describe which inputs push the figure up.

Metric	Value	Meaning
held-out ROC AUC	**0.8351** (min 0.80)	probability a random repeat is ranked above a random non-repeat
train ROC AUC · gap	0.8329 · -0.0022	no over-fitting
Brier score	0.1166	mean squared error of the probabilities
mean predicted vs actual share	0.2017 vs 0.2035	calibration check

The probability is rounded to 0.01 with exact arithmetic and banded: **HIGH** from 0.40, **MEDIUM** from 0.20, otherwise **LOW**. Tests confirm the likelihood rises when requirements fail and when conflicting works are added.

9.2 Expected crew hours — random forest

A `RandomForestRegressor` (300 trees, `min_samples_leaf=10` , `max_features=0.7`) predicts actual crew hours.

Metric	Value
held-out R²	**0.9025** (min 0.86)
train R² · gap	0.9181 · +0.0156
mean absolute error	0.4158 hours (vs 1.6806 h for always predicting the mean)
prediction range	0.4958 – 21.12 h

The estimate is rounded to 0.1 h. When planned hours are known, an overrun is expected if the estimate exceeds 1.3 × planned (compared exactly). The model file is 13.9 MB because of its 300 trees; this is the largest

artefact in the project.

9.3 Sentence type — TF-IDF + logistic regression

Sentences are split 80/20 with stratification (seed 1313). A TF-IDF vectoriser feeds a multinomial logistic regression.

Metric	Value
held-out macro F1	**0.9775** (min 0.95)
accuracy · weighted F1	0.9766 · 0.9766
train macro F1 · gap	0.9804 · +0.0029
weakest class	Location 0.9646, Hazard 0.9657

9.4 Training safeguards

- `run_model_training.py` refuses to save *any* model if one misses its minimum, so a half-updated model set can never be deployed.
- Every model is saved with its input schema and a metrics record (algorithm, rows, scores, minimum, scikit-learn version).
- Tests reload the saved files and re-score them, and check behaviour: likelihood rises with failures and conflicts; expected hours rise on a busy street.

CHAPTER 10

Model benchmarking

Ten alternative configurations, tuned by cross-validation, compared on the same held-out data

`experiment_models.py` benchmarks alternatives to each production model. Each candidate is wrapped in the same preprocessing pipeline, tuned with `GridSearchCV` using 5-fold cross-validation (stratified for the classifiers, seed 1313), refitted on all training data and scored on the held-out data with the same evaluation functions as production. Results are written to `output/model_benchmark_comparison.json`.



Figure 10.1 · Cross-validation and held-out score per candidate (the production model is dark)

10.1 Repeat likelihood

Candidate	CV ROC AUC	Held-out	Gap	Best parameters
HistGradientBoostingClassifier	0.8340	**0.8379**	+0.0128	l2_regularization=5.0, learning_rate=0.1, max_depth=5
LogisticRegression	0.8321	**0.8351**	-0.0019	C=0.1, class_weight=balanced
RandomForestClassifier	0.8299	**0.8358**	+0.0334	max_depth=None, min_samples_leaf=10, n_estimators=400

10.2 Expected crew hours

Candidate	CV R^2	Held-out	Gap	Best parameters
ExtraTreesRegressor	0.9017	**0.9080**	+0.0239	max_depth=None, min_samples_leaf=5, n_estimators=400
HistGradientBoostingRegressor	0.9038	**0.9075**	+0.0060	learning_rate=0.1, max_depth=4, max_iter=400, min_samples_leaf=20
RandomForestRegressor	0.9036	**0.9109**	+0.0333	max_depth=None, max_features=0.5, min_samples_leaf=3, n_estimators=400
Ridge	0.8593	**0.8703**	-0.0102	alpha=1.0

10.3 Sentence type

Candidate	CV macro F1	Held-out	Gap	Best parameters
Tfidf_CalibratedLinearSVC	0.9803	**0.9774**	+0.0032	C=0.25, ngram_range=[1, 2], sublinear_tf=True
Tfidf_ComplementNB	0.9793	**0.9750**	+0.0050	alpha=0.1, ngram_range=[1, 2]
Tfidf_LogisticRegression	0.9803	**0.9774**	+0.0030	C=1.0, class_weight=balanced, ngram_range=[1, 1], sublinear_tf=True

10.4 What the benchmark shows

- **Repeat:** gradient boosting edges ahead on AUC (0.838 vs 0.835) and calibration (Brier 0.116 vs 0.165 for the *balanced* logistic regression in the grid), but the gain is small. The production logistic regression is unweighted, which keeps its probabilities calibrated, and remains the most explainable choice. A calibrated gradient-boosting model is the leading candidate if a small accuracy gain is wanted.
- **Crew hours:** tuned tree ensembles all reach about 0.91; the linear Ridge baseline reaches only 0.87 and can predict negative hours, confirming the problem is non-linear. The tuned random forest (0.911) slightly beats production (0.903); gradient boosting has the smallest generalisation gap (0.006).
- **Sentence:** all three text models are within 0.003 of each other; the task is close to its ceiling with TF-IDF features.
- Every candidate clears every minimum, so the production choices are justified on simplicity, explainability and model size rather than on raw score alone.

CHAPTER 11

Knowledge base and retrieval

Sixteen PDFs as the system's rulebook and the chat's only source

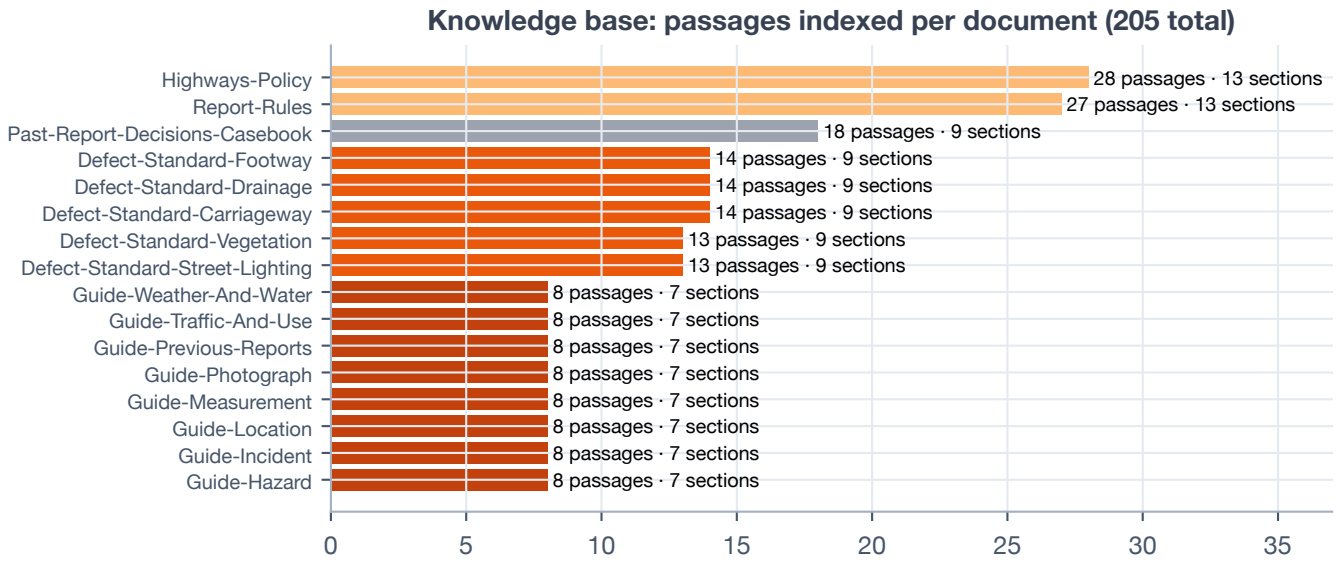


Figure 11.1 · Passages and sections per document

11.1 From PDF to passages

1. **Extract** text from each PDF with pypdf and remove the repeated page header (document name, issue, review cycle).
2. **Split into sections** at numbered headings (`1. Purpose and scope` …); sections must number 1…n, otherwise the document is refused — 136 sections in all.
3. **Cut into passages** of whole sentences up to 700 characters, carrying 120 characters over between passages so no sentence loses its context — 205 passages, each with `document_name` , `section_number` , `passage_number` and `carried_characters` .
4. **Embed and index** in a persistent ChromaDB collection using all-MiniLM-L6-v2 and cosine distance, with the HNSW search breadth set to the collection size so search is effectively exact.

11.2 Two ways the index is used

Use	How	Who uses it
Section read	fetch every passage of one section, order by passage number and re-join, dropping the carried-over characters — reconstructs the section exactly	tools: reviewer checks of a guide, what an inspection records, works order and resident update guidance, policy sections on triggers, decisions and documents
Semantic search	4 nearest passages per query, casebook excluded, cosine distance returned	chat: two searches per question (the question, and it with the previous question)

11.3 Requirements read from the standards

Defect type	Requirement	Rule read from the PDF

Carriageway	Location fixed	yes/no answer
Carriageway	Photograph	yes/no answer
Carriageway	Depth	minimum=40
Carriageway	Width	minimum=150, maximum=1500
Footway	Location fixed	yes/no answer
Footway	Photograph	yes/no answer
Footway	Depth	minimum=20
Footway	Width	minimum=150, maximum=1500
Drainage	Location fixed	yes/no answer
Drainage	Photograph	yes/no answer
Drainage	Water depth	minimum=50
Drainage	Water reach	minimum=1.0
Vegetation	Location fixed	yes/no answer
Vegetation	Highway land	yes/no answer
Vegetation	Clear height	footway_limit=2.4, carriageway_limit=5.1
Street lighting	Location fixed	yes/no answer
Street lighting	Column number	yes/no answer
Street lighting	Column lean	minimum=5.0

The standards are the council's authority. Reading them at run time — with a strict pattern per requirement kind and checks that each standard states its own defect type — means a policy change is made by updating the document, and a malformed document is refused rather than silently misread. A test plants a second set of standards with different figures and checks the screening follows them.

CHAPTER 12

Screening and business rules

The deterministic heart of the system

12.1 Requirement results

Each requirement reads one or two answers from the report. The result is **Met**, **Not met** or **Not documented** (a needed answer is blank). Follow-on questions are asked only when they apply: water reach only matters when water stands on the carriageway; column lean only when the fault is a leaning column. Boundaries are inclusive and compared exactly (“at least 40 mm” is met by 40).

12.2 Street works and coordination triggers

Trigger	Applies when
live street works	a **Live** permit on the report's street occupies the same highway part as the defect (carriageway / footway) — a *conflicting* permit
traffic-sensitive street	the defect type or linked type is carriageway or drainage, and the street category is not *Local road* (or is blank)
high column	street lighting, and the column height is blank or over 8 m

12.3 Assessment and open decisions

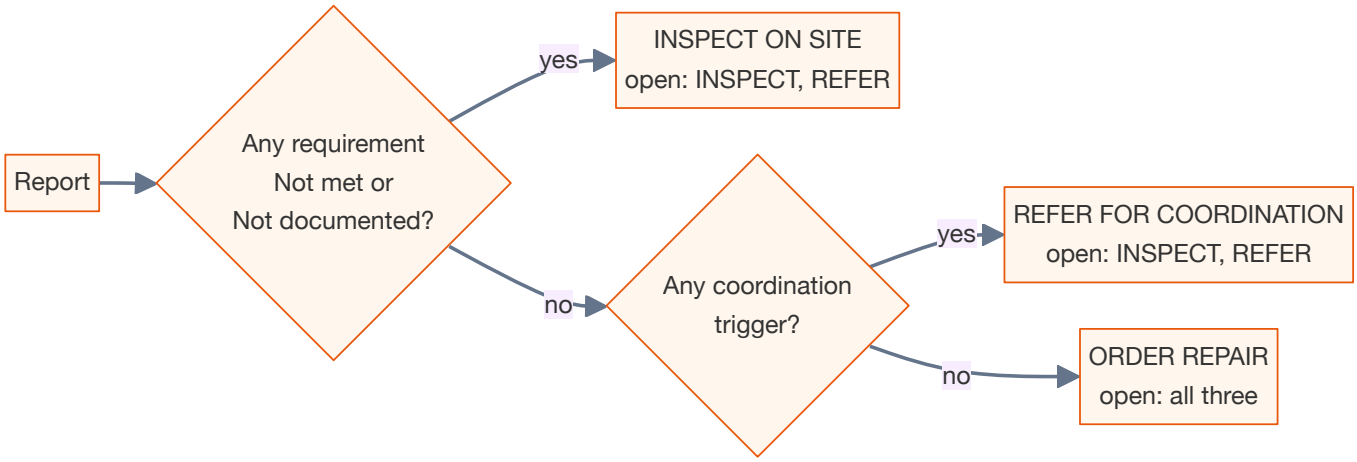


Figure 12.1 · Assessment rule and the decisions left open

The assessment is a recommendation the code makes; the inspector may choose any *open* decision. ORDER REPAIR is only open when every requirement is met and no trigger applies, because ordering a repair from an incomplete or conflicting report would send a crew out on bad information.

12.4 Statement evidence

Each attached document is split into sentences and classified by the sentence model. Each sentence is linked to the requirements its type bears on (Measurement → depth, width, water, clear height, lean; Location → location fixed, column number, highway land; Photograph → photograph; Weather and water → water depth and reach). Measurement sentences are scanned for figures; a figure that differs from the form is flagged

DISAGREES_WITH_FORM, and a figure for an answer left blank is flagged **DESCRIBES_BLANK_ANSWER**. Flags are evidence for the inspector — a document never replaces an answer on the report.

12.5 The queue

The open queue is ranked with reports whose defect type *or linked type* is footway first (trip hazards to pedestrians), then the rest, each group by reference. The captured queue has 40 reports; every row carries its screening counts, permits, triggers, assessment, likelihood band, expected hours, overrun flag and number of flagged sentences.

Assessment in the live queue	Reports
ORDER REPAIR	13
INSPECT ON SITE	18
REFER FOR COORDINATION	9

CHAPTER 13

The agentic review workflow

Seven agents, ten nodes, one durable pause

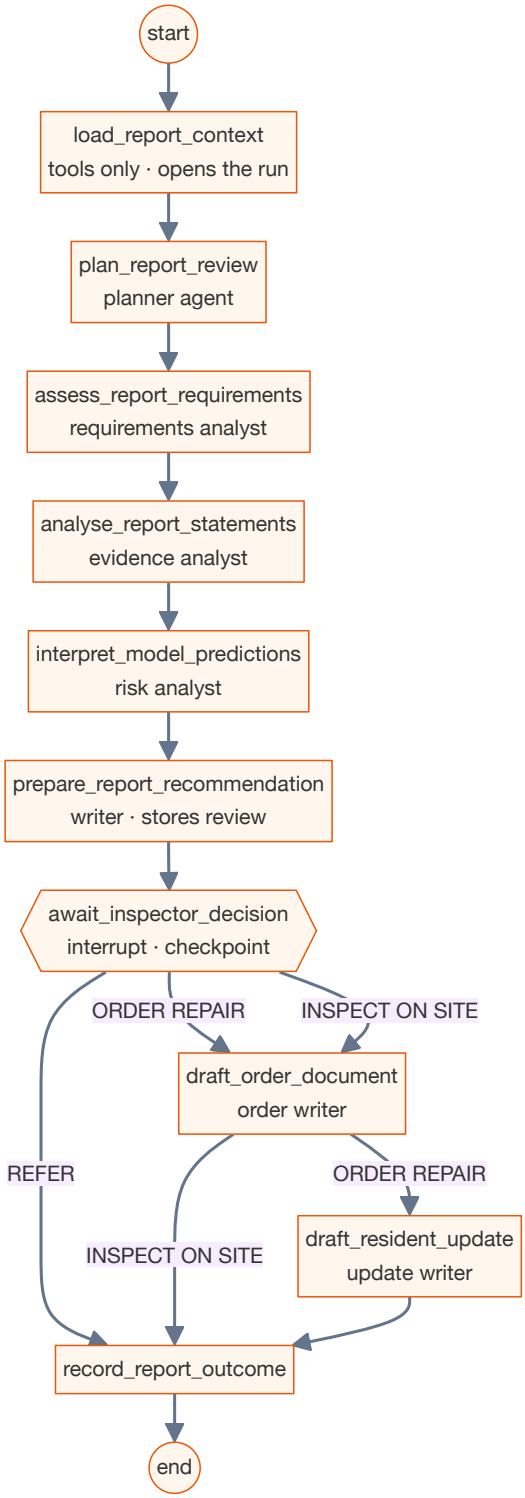


Figure 13.1 · The LangGraph state graph: two conditional branches, both chosen by the decision

13.1 State

`ReportReviewState` is a `TypedDict` of 23 keys: the request (reference, coordinator request and id, thread id), generated identifiers (run, review), the context from the tools (profile, documents, register count, screening,

evidence, flags, likelihood, crew hours), each agent's labelled fields, the decision, the documents and their details, and `llm_call_count`, which is *added to* by each node rather than replaced.

13.2 The agents

Agent	Reads	Writes (labelled fields)
review_planner_agent	report, documents, every requirement result, permits, triggers, assessment, the coordinator's request	PRIORITY · CHECKS · RATIONALE
requirements_analyst_agent	each requirement with the answers it reads, policy section on triggers, the plan's checks	REQUIREMENTS MET · NOT MET · NOT DOCUMENTED · COORDINATION TRIGGERS · SUMMARY
evidence_analyst_agent	every sentence with its type and requirements, every flag with both figures, each guide's reviewer checks	EVIDENCE FOUND · DISAGREEMENTS · CHECKS TO MAKE · SUMMARY
risk_analyst_agent	likelihood and band thresholds, expected hours and overrun ratio, every model input and how blanks are filled	LIKELIHOOD · EXPECTED CREW HOURS · PREDICTION SUMMARY
review_writer_agent	everything above plus the policy section on the three decisions and the decisions open	RECOMMENDATION · RECOMMENDED DECISION
order_writer_agent	decision, recipient, details built by code, guidance from the standards or policy	SUBJECT · DOCUMENT
resident_update_writer_agent	works order details, each standard's resident update guidance	HEADLINE · UPDATE

13.3 Nodes

Node	Type	What it does	AI calls	Writes
load_report_context	tools	gathers the profile, documents, permits, screening, evidence, flags, likelihood and crew hours; opens the run	0	agent_runs (RUNNING)
plan_report_review	agent	planner sets the priority and the checks for this report	1	llm_calls
assess_report_requirements	agent	requirements analyst explains each result and the triggers, using the policy section on triggers	1	llm_calls
analyse_report_statements	agent	evidence analyst reviews every sentence and flag with the guides' reviewer checks	1	llm_calls
interpret_model_predictions	agent	risk analyst explains the likelihood band and the expected crew hours	1	llm_calls
prepare_report_recommendation	agent	writer recommends one open decision; the review is stored in one transaction	1	report_reviews, llm_calls, agent_runs (AWAITING_DECISION)
await_inspector_decision	gate	pauses (interrupt + checkpoint) until an inspector decides; re-checks the decision on resume	0	checkpoint.db
draft_order_document	agent	works order (ORDER) or inspection request (INSPECT), with details built by code	1	llm_calls
draft_resident_update	agent	update for the resident after a works order	1	llm_calls

record_report_outcome	tools	decision, documents, coordination brief and the completed run in one transaction	0	review_decisions, report_reviews, agent_runs (COMPLETE)

13.4 Decision making

Decision	Open when the assessment is	Path after the gate	Status recorded	AI calls
ORDER REPAIR	ORDER REPAIR only	works order → resident update → record	REPAIR_ORDERED	7
INSPECT ON SITE	any assessment	inspection request → record	INSPECTION_REQUESTED	6
REFER FOR COORDINATION	any assessment	record + coordination brief built by code	REFERRED_FOR_COORDINATION	5

Who may decide	Checked before the run resumes (refused with)
a Highways Inspector appointed for the street's district	another role (403) · another district (403)
who did not open the review	own review (409)
choosing an open decision with a reason of 10–1000 characters	decision not open (409) · bad request shape (422)
once per review	already decided (409) · a different decision still being recorded (409)

Every check runs before any AI call is spent, so a refused decision costs nothing and changes nothing.

CHAPTER 14

The street care chat

One question, one model call, answers only from the documents or the records

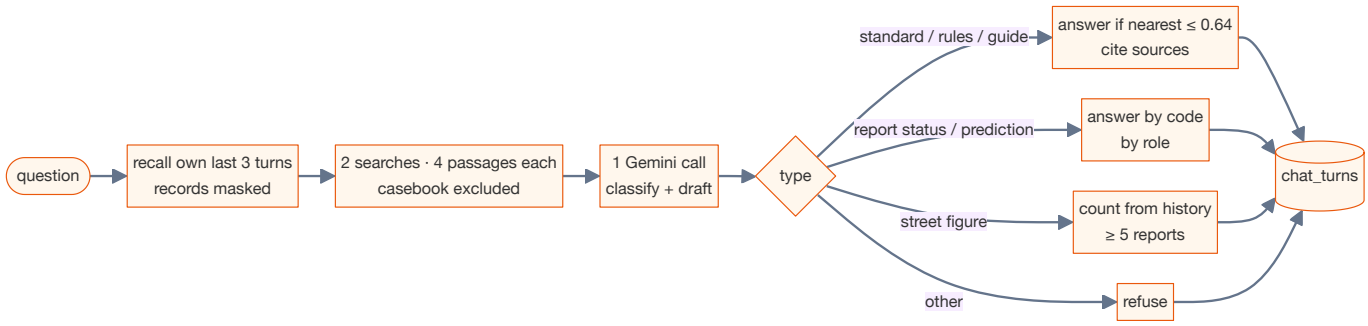


Figure 14.1 · The chat pipeline

14.1 Classification and grounding

The model classifies each question into one of seven types — DEFECT STANDARD, STREET CARE RULES, STATEMENT GUIDANCE, REPORT STATUS, OUTCOME PREDICTION, STREET FIGURE, OUT OF SCOPE — and drafts an answer. Document questions are answered only when the nearest passage is within cosine distance 0.64; otherwise they are refused. The reply's report reference must appear in the question or the recent conversation, so the model cannot invent one.

14.2 Answers from the records, by role

Asker	Report status	Outcome prediction
Customer Services Adviser	“SC0052125 (Footway) on Market Place is awaiting a decision.” Once a repair is ordered: where on the street, and the resident update text	always: “customer services is not told whether a defect is likely to come back or how long its repair will take”
Inspector / Coordinator	a six-line record: review and status; screening; permits (conflicting marked); figures; statement evidence; recommendation or decision	the same six-line record

14.3 Privacy

Street figures count distinct decided reports of one defect type on one street. Below five reports the figure could identify a household, so everyone hears the group-size line instead. Captured examples: High Street footway → “801 footway defects have been reported on High Street”; Kiln Lane footway (2 reports) → the privacy line.

14.4 Memory

The asker's own last three turns are replayed oldest first; answers that came from the records are replaced with a fixed note, so nothing from the review records ever reaches a model prompt. Nobody else's conversation is ever recalled.

CHAPTER 15

Persistence and data model

Six tables for the application, two for the workflow checkpoints

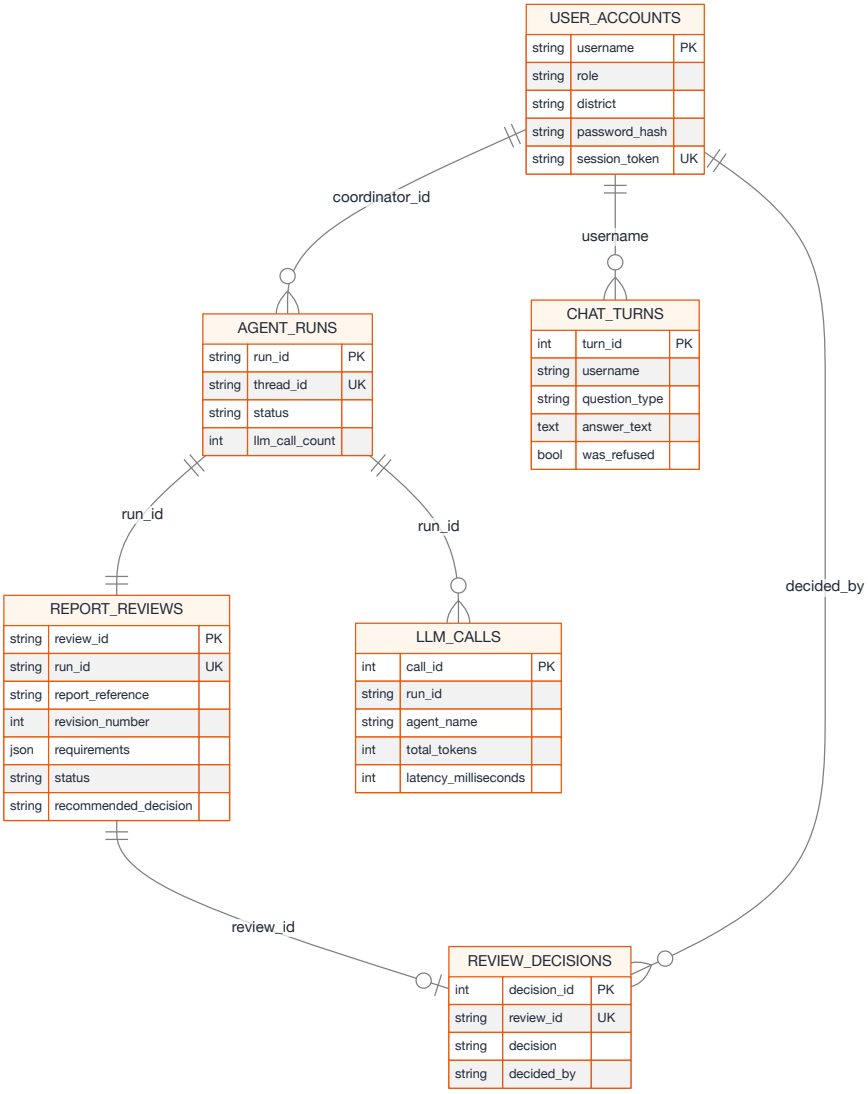


Figure 15.1 · Application database (logical links; no enforced foreign keys)

Table	Rows mean	Notes
user_accounts	a staff account	PBKDF2-SHA256, 200,000 rounds, 16-byte salt per account; one unique session token
agent_runs	one review run	RUNNING → AWAITING_DECISION → COMPLETE, or FAILED with the calls recorded
report_reviews	one review of one report revision	38 columns: screening, permits, triggers, evidence and flags (JSON), figures, five write-ups, documents, brief
review_decisions	the decision on a review	unique per review
llm_calls	one model call	agent, model, prompt / response / total tokens, latency in ms
chat_turns	one chat question	type, report / street / defect type, answer, sources, closest distance, refused

The schema is declared once in `persistence/models.py` (SQLAlchemy ORM) and created by `run_database_setup.py`; every read and write goes through `persistence/repository.py`. The checkpoint database is owned by LangGraph (`checkpoints` and `writes` tables). The links are enforced in code, not as database foreign keys — acceptable for a single-service SQLite deployment, and noted as an improvement for a multi-service one.

CHAPTER 16

REST API and security

Eleven endpoints behind one guard

Method · path	Permission	Success	Calls
POST /auth/signin	public	200 + token, pages, permissions	0
POST /auth/signout	token	200	0
GET /queue?defect_type=	queue	200 ranked rows	0
GET /queue/{{report_reference}}	queue	200 detail	0
POST /reviews	review	201 review at the gate	5
GET /reviews?status_value=	audit	200 reviews	0
POST /reviews/{{id}}/decision	decision	200 decided review	0–2
GET /reviews/{{id}}/calls	audit	200 call budget	0
POST /chat/ask	chat	200 answer	1
GET /chat/turns?turn_limit=	chat	200 own turns	0
GET /audit/chat-turns?turn_limit=	audit	200 everyone's turns	0

16.1 Security model

1. **Authentication.** Sign-in compares the salted PBKDF2 hash in constant time and answers an unknown user and a wrong password identically. A fresh 24-byte URL-safe token replaces any earlier one, so an account holds one session.
2. **Authorisation.** One dependency, `authorize_request`, sits on the whole guarded router, so a new endpoint cannot forget it. It maps the method and route template to a permission and checks the role holds it — before the endpoint runs, even for things that do not exist.
3. **District rule.** `authorize_district` holds a Highways Inspector to their own district's streets, before any run starts or any decision is looked at.
4. **Validation.** Pydantic schemas refuse bad shapes with 422 before any work: text lengths, the six queue filters, the four statuses, the three decisions, turn limits 1–200.
5. **Failure handling.** One handler turns any `ValueError`, Gemini error or HTTP error into a 502 with the failure's own message, and marks a run the request opened as FAILED.
6. **Concurrency.** A lock per report stops the same report being reviewed twice, or a review decided twice, at once.

Code	Example (captured)
401	Username or password is not correct · Sign in first
403	The Customer Services Adviser role does not include the queue permission priya is appointed for North and may not act on report SC0052125, which is on Market Place
404	No open defect report has the reference SC0000000
409	Revision 1 of report SC0052125 has already been reviewed; each revision is reviewed once amelia opened the review of report SC0052125 and may not decide it ORDER REPAIR is not open for report SC0052186: a requirement is Not met or Not documented

422	field-level list, e.g. String should have at least 10 characters
502	the model or validation failure's own message; run marked FAILED, nothing stored

CHAPTER 17

The dashboard

Five pages over the service, shaped by role

Page	Who	What it shows and does
Report queue	Inspector, Coordinator	counts (reports, awaiting review / decision, footway, each assessment); filter by defect type; table with status, assessment, requirement counts, permits, triggers, flags, documents, likelihood + band, expected hours, overrun; selecting a row opens it in the detail
Report detail	Inspector, Coordinator	headline figures; tabs for Screening (requirements, blank answers, decisions open, triggers, permits), Statement evidence (every sentence, flags with both figures), Report (answers and what each is used for) and Review (recommendation, decision); start a review (5–1000 characters)
Decision queue	Inspector	the inspector's district and streets; reviews awaiting a decision there; recommendation, figures, requirements, triggers, permits, flags, write-ups; a decision form offering only the decisions open, with a 10–1000 character reason, hidden on one's own review
Audit trail	Inspector, Coordinator	reviews by status; full record, documents, coordination brief; call budget (calls vs expected, tokens, latency) and every call; latest 200 chat turns from everyone
Street care chat	everyone	own conversation (latest 20), badged by type with sources; ask box (3–1000 characters)

17.1 Design rules

- The dashboard holds no business rule. Every figure comes from the service, so a browser, a script and the tests see the same answer.
- It repeats only the service's own input limits, an inspector's own district and the own-review rule, so a form the service would refuse is never sent.
- All requests go through one client (`call_service`) that adds the token, shows a timed spinner only for calls that cost model calls, ends the session on a 401 and shows failures in the configuration's words.
- Navigation is built from the pages the service returned for the role; until sign-in, only the sign-in page exists.
- Badges are colour-coded by decision, status, likelihood band, chat type, requirement result and overrun.

Every page was rendered against the running service with Streamlit's AppTest: sign-in, queue, detail, starting a review, the decision queue as the appointed inspector (recording “SC0052109 is now inspection requested …”), the audit trail and the chat — with no exceptions.

CHAPTER 18

Testing and quality assurance

Fifty tests against the real data, models, service and language model

Area	Tests	Needs Gemini
Day 1 · cleaning	7	no
Day 2 · documents & screening	15	no
Day 3 · features & models	13	no
Day 4 · agents (offline) & street figures	2	no
Day 4 · review workflow run	8	yes
Day 4 · chat	2	yes
Day 5 · service & access	3	no
Total	**50**	**10**

18.1 What the tests check

- **Cleaning:** rebuilding a cleaned file reproduces it exactly; planted rows check revisions, units (e.g. 1.75 in → 44 mm), canonical references, impossible readings and refused keys.
- **Knowledge and screening:** every document yields text; sections and passages are complete; section reads are exact; each defect type is read from its own standard — including from planted standards with different figures; boundaries, follow-on questions, linked types, triggers and the worked example of the specification.
- **Models:** saved schemas exclude outcomes and identifiers; scores above minimums; behaviour (likelihood rises with failures and conflicts; hours rise on busy streets); bands and overrun thresholds.
- **Workflow:** agents called once, in order, with token and latency records; the stored review equals the tools' findings; the order writes both documents; graph paths and resume from a checkpoint on a fresh build of the workflow; refusals before any call; one decision only.
- **Service and chat:** sign-in and one-session rule; every role × permission probe; queue order and row/detail agreement; chat grounding and citations; memory isolation; adviser and team answers; street figures need five reports.

18.2 Results

Run	Result
Full suite with a Gemini key (or a stand-in model)	**50 passed** in about 24 s
Full suite without a key	**40 passed, 10 failed** — exactly the 10 that call Gemini; each fails with the service's 502 “API key not valid”
Fresh project from the Day 1 commit + 20 zips	all four build steps succeed; row counts and scores reproduced exactly
Captured service run	76 requests, every status code as specified (see Appendix B)
Static analysis	pyflakes reports no issues

18.3 Issues encountered and resolved

Issue encountered	Resolution
Chat turns stored `source_documents` as JSON while the tests store comma-joined text	column changed to `Text`; database rebuilt with `run_database_setup.py`
Floating-point rounding turned 3.95 cm into 39 mm instead of 40 mm	all unit conversion and threshold comparison moved to `Decimal` with ROUND_HALF_UP
Paused review runs had to survive a restart and be resumed from another request thread	LangGraph `SqliteSaver` on a SQLite connection opened with `check_same_thread=False`
Requirement rules are `Decimal` values, which are not plain JSON for the API	rules converted to numbers in the report detail; stored reviews keep requirements without rules
The permission guard needs the route template, not the URL	permission looked up from the matched route's path template
Agents decorate replies (`**PRIORITY:**` , `## Checks:` , repeated labels)	tolerant labelled-reply parser; edge decoration stripped before checking a recommended decision
Approximate vector search could miss a passage of a section	HNSW search breadth set to the collection size, making search effectively exact
Gemini key not configured: 10 tests failed with 502 (API key not valid)	setup documented; workflow verified end to end with a stand-in model; all 50 pass with a key
`requirements.txt` lacks `sentence-transformers` , so indexing fails on a fresh install	installed separately in the setup steps; to be added to the file
Five developers editing in parallel risked merge conflicts	one owner per file per day; the one cross-day fix (`persistence/models.py`) shipped in a single Day 4 branch
Generated databases, models and CSVs were committed to git	flagged for removal; everything is reproducible with the four build steps

CHAPTER 19

Delivery plan and team organisation

Five days, five developers, one file per owner

The build was split into five themed days. Each day was divided between the five developers so that every file had exactly one owner; each developer pushed an individual branch and the five branches of a day merged without conflicts.

Day	Theme	What the day delivers	Tests passing after the day
1	**Data**	database and accounts, the six cleaners, quality profiler, data pipeline	7
2	**RAG**	PDF extraction, sections and passages, vector index, standard reading, screening	22
3	**ML**	feature builder, three models, evaluation and registry, evidence and prediction tools	35
4	**Agents**	Gemini client, seven agents, ten nodes, LangGraph workflow, chat	—
5	**API and Streamlit**	permissions, schemas, eleven endpoints, service runner, dashboard	50 (with the Gemini key)

Five-day plan by role

Day	Developer 1	Developer 2	Developer 3	Developer 4	Developer 5
1 · Data	persistence: database, models	repository, password security, database setup	loader, common cleaning	report, sentence, document and register cleaning	history and repair log cleaning, profiler, pipeline
2 · RAG	PDF extraction, chunking	vector index, retrieval tools, indexing	standard reading	report data tools	screening tools
3 · ML	feature builder	registry, evaluation, review documents	repeat and crew hours models, training	sentence model, evidence tools	prediction and decision tools
4 · Agents	state, language model	agents	graph	nodes	chat (three modules)
5 · API and Streamlit	permissions, schemas	routes	service runner, API client, sign-in	queue and detail pages, dashboard runner	decision queue, audit trail, chat page

Team structure

Developer	Team member	Areas owned across the five days
Developer 1	Mansavi Aggarwal	persistence models, PDF extraction and chunking, feature builder, workflow state and Gemini client, permissions and API schemas
Developer 2	Puttamaneni Srinivasa Praneeth	repository and accounts, vector index and retrieval, model registry and evaluation, the seven agents, the REST routes

Developer 3	Riya Rajeev Shinde	loader and common cleaning, standard reading, repeat and crew hours models, the workflow graph, service runner and sign-in
Developer 4	Bhavesh Durai S	report, sentence, document and register cleaning, report data tools, sentence model and evidence, workflow nodes, queue and detail pages
Developer 5	Paravthy Vinod K V	history and repair log cleaning and the pipeline, screening tools, prediction and decision tools, the chat, decision queue, audit trail and chat page

CHAPTER 20

Deployment and operations

How to build, run, monitor and reset the system

20.1 Setup and build

```
cd Project
python -m venv venv && source venv/bin/activate     # Windows: venv\Scripts\Activate.ps1
pip install -r requirements.txt && pip install sentence-transformers
# Project/.env:  GEMINI_API_KEY=…   GEMINI_MODEL=…
python run_database_setup.py      # 6 tables, 16 accounts          (<1 s)
python run_data_pipeline.py       # 105,808 clean rows + report    (~4 s)
python run_model_training.py      # 3 models, metrics              (~10 s)
python run_knowledge_indexing.py  # 205 passages                   (~14 s, downloads model once)
python -m pytest tests.py -q      # 50 passed
python run_api_service.py         # http://localhost:8081/docs
streamlit run run_dashboard.py    # http://localhost:8080
```

20.2 Operational behaviour

Situation	Behaviour
A build output is missing	`run_api_service.py` lists every missing path with the step that produces it and exits 1
Gemini unavailable or key wrong	queue, detail, screening and predictions keep working; reviews and chat return 502; the run is FAILED and can be retried
Writing a document fails after a decision	the checkpoint holds the decision; sending the same decision again finishes the run; anyone else is refused (409)
Service restarts while reviews wait	paused runs resume from `checkpoint.db`
A model misses its minimum on retraining	nothing is saved; the previous models stay in place
Tests run out of unreviewed ORDER reports	delete `streetcare.db` and `checkpoint.db`, run database setup again

20.3 Monitoring

The audit trail shows, per review, the calls recorded against the calls expected, total tokens and latency, and every call. `llm_calls` supports cost reporting by agent and model; `agent_runs` shows FAILED runs to investigate; `chat_turns` shows refusal rates and closest distances, which indicate whether the documents cover what staff ask.

20.4 Configuration changes

Thresholds that come from the council's documents (requirement figures) change by updating the PDFs and re-indexing. Operational settings (likelihood bands, overrun ratio, chat distance, group-size minimum, roles, districts and streets, messages, ports) change in `config.py`. After changing cleaning rules, re-run the pipeline and training: the tests check the cleaned files match what the current code produces.

CHAPTER 21

Limitations, risks and future work

What to watch, and what to improve next

21.1 Limitations

- **Synthetic scenario data.** The extracts are realistic but constructed; scores on live council data must be re-measured.
- **Static queue.** The queue and details are computed once at start-up; new reports need a restart (or a reload endpoint).
- **SQLite.** Fine for one service instance; concurrent multi-instance deployment needs PostgreSQL and enforced foreign keys.
- **Language model variability.** Wording varies between runs; validation catches malformed replies but a refused reply costs a retry.
- **Calibration.** The repeat model is ranked well (AUC 0.835) but the likelihood is a band, not a guarantee.

21.2 Risks and mitigations

Risk	Impact	Mitigation
Gemini key missing (no `.env` in the repository)	reviews and chat fail; 10 tests fail	document the setup step; keep the key out of git; consider a startup check
`sentence-transformers` not in requirements	knowledge indexing fails on a fresh install	add it to `requirements.txt`
Generated databases, models and processed CSVs committed to git	repository bloat (13.9 MB model), merge conflicts, stale data	remove from git and rely on `.gitignore`; rebuild with the four steps
Prompt injection through report documents	an agent could be steered	agents only explain; every figure, rule and decision is code; replies are validated; a person decides
Model drift	predictions degrade	retrain on new history; training refuses to save below the minimums
Privacy of residents	re-identification from small counts	group-size rule of five; advisers see no predictions; chat memory masks record answers

21.3 Future work

1. Replace the production repeat model with a calibrated gradient-boosting model if the small AUC and Brier gains are wanted (Chapter 10).
2. Add SHAP-style explanations so the risk analyst can name the strongest inputs for each report.
3. Reload the queue on a schedule or on demand; connect to a live street works feed.
4. Move to PostgreSQL with foreign keys; containerise the service and dashboard.
5. Add structured logging and cost dashboards from `llm_calls`; alert on FAILED runs.
6. Evaluate the agents' write-ups with a rubric (faithfulness to the tools, completeness) as part of the test suite.

CHAPTER 22

Conclusion

StreetCare AI meets its brief. A reproducible build turns messy forms into 105,808 trusted rows, three models that clear their minimums (ROC AUC 0.8351, R^2 0.9025, macro F1 0.9775), and a knowledge base of 205 passages whose standards define the rules at run time. Every report is screened by code; five agents explain the findings and recommend; a durable pause ensures the right Highways Inspector decides; and the documents each decision calls for are written and checked before anything is stored. The service secures every route, the dashboard gives each role what it needs and nothing more, and the chat answers from the documents while protecting predictions and residents' privacy.

The central design choice — **code decides the facts, AI explains them, a person makes the decision** — makes the system auditable, testable and safe to run, and is the main lesson for similar public-service AI systems. With a Gemini key configured, the dependency list completed and generated files removed from version control, the system is ready for a pilot with the Street Care Office.

APPENDIX A

Configuration reference

The most important settings in config/config.py

Setting	Value	Meaning
NULL_TOKENS	'', n/a, na, -, null, not known, caller unsure	values read as blank
REPORT_REFERENCE_PREFIX / DIGITS	SC / 7	canonical reference
RANDOM_SEED	1313	splits and models
SENTENCE_HELD_OUT_SHARE	0.2	sentence model test share
MINIMUM_SENTENCE_WORDS	3	shorter sentences dropped
LOGISTIC_REGRESSION_MAX_ITERATIONS	3000	repeat and sentence models
CREW_HOURS_TREE_COUNT / MIN_SAMPLES_LEAF / MAX_FEATURES	300 / 10 / 0.7	random forest
MINIMUM_REPEAT_ROC_AUC / CREW_HOURS_R2 / SENTENCE_MACRO_F1	0.80 / 0.86 / 0.95	training refuses to save below these
HIGH / MEDIUM_LIKELIHOOD_MINIMUM	0.40 / 0.20	likelihood bands
OVERRUN_RATIO	1.3	expected overrun
HIGH_COLUMN_TALLER_THAN_METRES	8	high column trigger
CHUNK_SIZE_CHARACTERS / OVERLAP	700 / 120	passages
EMBEDDING_MODEL_NAME	all-MiniLM-L6-v2	embeddings, cosine
CHAT_RETRIEVAL_RESULT_COUNT / MAX_RELEVANCE_DISTANCE	4 / 0.64	chat search and grounding
CHAT_HISTORY_TURN_LIMIT / PAGE / AUDIT	3 / 20 / 200	memory and page sizes
GROUP_SIZE_MINIMUM	5	street figure privacy
LLM_CALLS_BEFORE_DECISION / BY_DECISION	5 / ORDER 7, INSPECT 6, REFER 5	call budget
GEMINI_MAX_OUTPUT_TOKENS / TEMPERATURE	8192 / 0.2	generation
IDENTIFIER_HEX_LENGTH	12	run, review and thread ids
PASSWORD_HASH_ALGORITHM / ROUNDS / SALT_BYTES	sha256 / 200,000 / 16	PBKDF2
SESSION_TOKEN_BYTES / HEADER	24 / X-Session-Token	sessions
COORDINATOR_REQUEST / DECISION_REASON / CHAT_QUESTION length	5–1000 / 10–1000 / 3–1000	input limits
API_HOST / API_PORT	0.0.0.0 / 8081	service
Dashboard port	8080 (.streamlit/config.toml)	dashboard

APPENDIX B

API reference with examples

Captured requests and replies (AI wording from a stand-in model where marked)

Sign in 200

```
POST /auth/signin
{"username": "grace", "password": "22222222"}
```


```
{
 "username": "grace",
 "display_name": "Grace Turner",
 "role": "Street Care Coordinator",
 "district": null,
 "session_token": "…32 characters…",
 "pages": [
  "Report queue",
  "Report detail",
  "Audit trail",
  "Street care chat"
 ],
 "permissions": [
  "queue",
  "review",
  "audit",
  "chat"
 ]
}
```


Queue with an unknown defect type 422

```
GET /queue?defect_type=Graffiti
```


```
{
 "detail": [
  {
   "type": "literal_error",
   "loc": [
    "query",
    "defect_type"
   ],
   "msg": "Input should be 'All', 'Carriageway', 'Footway', 'Drainage', 'Vegetation' or 'Street lighting'",
   "input": "Graffiti",
   "ctx": {
    "expected": "'All', 'Carriageway', 'Footway', 'Drainage', 'Vegetation' or 'Street lighting'"
   }
  }
 ]
}
```

Team asks for a report's status while the review waits 200

```
POST /chat/ask
{"question": "What is the status of SC-52125?"}
```


```
{
 "question_type": "REPORT STATUS",
 "report_reference": "SC0052125",
 "street_name": null,
 "defect_type": null,
 "answer": "REV-ABDB38840557, report SC0052125 revision 1 (Footway) on Market Place, opened by amelia: awaiting a decision.\nAssessment ORDER REPAIR. Footway: Location fixed Met, Photograph Met, Depth Met, Width Met. Coordination triggers: none. The decisions open: ORDER REPAIR, INSPECT ON SITE, REFER FOR COORDINATION.\nStreet works permits live on the street: SWP0071384.\nRepeat likelihood 0.03 (LOW). Expected 3.1 crew hours against 2.5 planned: no overrun is expected.\nStatement evidence: 9 sentences read (Hazard, Traffic and use, Background, Measurement, Photograph); statement sentences flagged against the report: 0.\nIt waits for a Highways Inspector's decision; the review recommends ORDER REPAIR.",
 "source_documents": [],
 "was_refused": false
}
```


Adviser asks after the repair was ordered (update text: stand-in wording) 200

```
POST /chat/ask
{"question": "What is the status of report SC0052125?"}
```


```
{
 "question_type": "REPORT STATUS",
 "report_reference": "SC0052125",
 "street_name": null,
 "defect_type": null,
 "answer": "SC0052125 (Footway) on Market Place is repair ordered. Where on the street: opposite no. 50. What the resident has been told: Thank you for your report. A repair has been ordered. - Report reference: SC0052125 - Defect type: Footway - Linked defect type: No linked defect - Street: Market Place - Location: opposite no. 50 Street Care Office, Wrenford Borough Council",
 "source_documents": [],
 "was_refused": false
}
```


Street figure 200

```
POST /chat/ask
{"question": "How many footway defects have been reported on High Street?"}
```


```
{
 "question_type": "STREET FIGURE",
 "report_reference": null,
 "street_name": "High Street",
 "defect_type": "Footway",
 "answer": "801 footway defects have been reported on High Street.",
 "source_documents": [],
 "was_refused": false
}
```

Street figure below the group size 200

```
POST /chat/ask
{"question": "How many footway defects have been reported on Kiln Lane?"}
```


```
{
 "question_type": "STREET FIGURE",
 "report_reference": null,
 "street_name": "Kiln Lane",
 "defect_type": "Footway",
 "answer": "Figures about defects on a street are shared only where at least five reports make them up.",
 "source_documents": [],
 "was_refused": false
}
```


Inspector of another district decides 403

```
POST /reviews/REV-ABDB38840557/decision
{"decision": "ORDER REPAIR", "decision_reason": "Every requirement of the standard is met on the
report and no coordination trigger applies."}
```


```
{
 "detail": "priya is appointed for North and may not act on report SC0052125, which is on Market Place"
}
```


Call budget after ORDER REPAIR 200

```
GET /reviews/REV-ABDB38840557/calls
```


```
{
 "review_id": "REV-ABDB38840557",
 "run_id": "RUN-2BC6397CA974",
 "call_count": 7,
 "expected_call_count": 7,
 "total_tokens": 4218,
 "total_latency_milliseconds": 7,
 "calls": [
  "review_planner_agent",
  "requirements_analyst_agent",
  "evidence_analyst_agent",
  "risk_analyst_agent",
  "review_writer_agent",
  "order_writer_agent",
  "resident_update_writer_agent"
 ]
}
```

APPENDIX C

File inventory

Every module and what it does

File	Purpose
`data_pipeline/dataset_loader.py`	read raw as text with null tokens; save and load processed
`data_pipeline/common_cleaning.py`	shared cleaning steps (13 functions)
`data_pipeline/history_cleaning.py · repair_log_cleaning.py · sentence_cleaning.py`	the three training cleaners
`data_pipeline/report_cleaning.py · document_cleaning.py · register_cleaning.py`	the three live cleaners
`data_pipeline/quality_profiler.py`	data quality report
`data_pipeline/feature_builder.py`	requirement counts, trigger flags, feature frame, model pipeline
`modeling/repeat_model.py · crew_hours_model.py · sentence_model.py`	trainers
`modeling/model_evaluation.py · model_registry.py`	scores; save and load models, schemas, metrics
`knowledge/pdf_extraction.py · chunk_splitting.py`	PDF text; sections and passages
`knowledge/vector_index.py`	build, open, read sections, search
`knowledge/standard_reading.py`	parse each defect standard's requirements
`tools/report_data_tools.py`	load live files, fetch, rank, describe
`tools/screening_tools.py`	street works count, triggers, requirement results, assessment
`tools/evidence_tools.py`	sentence evidence, links, disagreements
`tools/prediction_tools.py`	likelihood with band, expected hours with overrun
`tools/retrieval_tools.py`	read named sections of standards, guides, policy
`tools/decision_tools.py`	review conflict and decision refusal
`tools/review_document_tools.py`	works order and inspection details, coordination brief
`persistence/database.py · models.py · repository.py`	engine; 6 ORM tables; all reads and writes
`access/password_security.py · permissions.py`	PBKDF2; request and district guards
`agentic/state.py · language_model.py`	workflow state; Gemini client and reply parser
`agentic/agents.py · nodes.py · graph.py`	7 agents; 10 nodes and router; compiled graph with checkpointer
`chat/conversation_memory.py · street_care_assistant.py · record_answer.py`	memory; classify and answer; answers from the records
`api/schemas.py · routes.py`	Pydantic shapes; 11 endpoints, guard, failure handler
`dashboard/*`	API client, sign-in, five pages
`run_*.py`	four build steps, service runner, dashboard entry
`experiment_models.py`	benchmark of alternative models
`tests.py`	50 tests

APPENDIX D

Glossary

Term	Meaning
Assessment	the decision the screening recommends: ORDER REPAIR, INSPECT ON SITE or REFER FOR COORDINATION
Decisions open	the decisions an inspector may choose for a report
Requirement	a rule in a defect standard, e.g. a minimum depth; result Met, Not met or Not documented
Coordination trigger	live street works, traffic-sensitive street or high column
Conflicting permit	a Live street works permit on the same street and highway part as the defect
Linked defect type	a second defect type at the same location, screened under its own standard too
Repeat likelihood	predicted probability the defect is reported again after repair, banded HIGH / MEDIUM / LOW
Expected crew hours	predicted actual hours; overrun when above 1.3 × planned
Sentence type	one of nine classes for a document sentence
Flag	a document figure that disagrees with the form, or describes a blank answer
Passage	a chunk of up to 700 characters of whole sentences from a PDF section
RAG	retrieval-augmented generation: answering from retrieved passages
Agent	one language model call with a fixed role, prompt and labelled reply
Gate / interrupt	the workflow's pause until an inspector decides
Checkpoint	the saved workflow state that lets a paused run resume, even after a restart
Thread id	the identifier of a run's checkpoint: report reference + 12 hex characters
ROC AUC	how well a classifier ranks positives above negatives (0.5 random, 1.0 perfect)
R^2	share of variance a regressor explains
Macro F1	the unweighted mean of per-class F1 scores
Brier score	mean squared error of predicted probabilities
Generalisation gap	training score minus held-out score; large means over-fitting
PBKDF2	a slow, salted password hashing method